Qubit & Toffoli Reduction Techniques

A Comprehensive Primer

ecdsa.fail quantum circuit benchmark

Contributors

The optimization techniques in this primer were collectively contributed by **all participants and leaderboard contributors of the ecdsa.fail challenge**.

Summarized and compiled by **Jieyi Long**, with Claude Code.

Audience note

This primer is written for undergraduate students who have just learned the basics of quantum computing — superposition, entanglement, Toffoli/CNOT/Hadamard gates, quantum circuits, and reversible computation. It explains every optimization technique used in the ecdsa.fail challenge from circuit-level first principles, with concrete examples from actual research.

As of 2026-06-27

Contents

1. Introduction: What Is the ecdsa.fail Challenge?

Elliptic-curve cryptography (ECC) secures essentially all of today's digital signatures — including the keys that control Bitcoin and Ethereum funds. A large enough quantum computer would break it: **Shor's algorithm** can recover a private key from a public key by solving the elliptic-curve discrete logarithm problem in time polynomial in the key size. The overwhelming majority of that quantum computation is spent running **one tiny primitive over and over** — *elliptic-curve point addition* on the curve `secp256k1` (Bitcoin's curve). Make that inner circuit cheaper and the entire attack gets cheaper. **ecdsa.fail** is an open, collaborative competition to build the **leanest possible reversible quantum circuit for that single point addition** — a public, reproducible race to map exactly how expensive (or cheap) breaking ECC really is.

A submission is a circuit that computes one in-place affine point addition $P \mathrel{+}= Q$ on secp256k1, and it is scored by its **spacetime cost: `peak_qubits`×`average_executed_Toffoli`** (lower is better) — the two resources that dominate cost on a fault-tolerant quantum computer (§2). Every submission must be *exactly correct and physically valid*: it is checked against 9,024 random test cases for classical-value correctness, for reversibility (all scratch qubits returned to $|0\rangle$), and for phase cleanliness, and contributors may only edit the point-addition circuit itself — the evaluation harness, validator, and toolchain are locked. The headline goalpost is **Google Quantum AI's published resource estimate** for the same task (≈1,425 qubits × ≈2.1M Toffoli ≈ 2.99×10^9 qubit-Toffoli, March 2026). The challenge's starting baseline scored $\sim 1.07 \times 10^{10}$; as of late June 2026 the community-driven state of the art sits near **1,152 qubits × ∼1.36M Toffoli** ≈ 1.57×10^9 — roughly **47% below** Google's number, with the bar still dropping.

What makes the effort unusual is *how* the records are won. Participants — ranging from academic researchers to programmers with no quantum-computing background — increasingly drive the work with **AI agent pipelines**: the circuit harness is fed directly into large-language-model loops that propose, test, and validate optimizations around the clock, preserving their findings as versioned research notes. The result is a fast-moving catalogue of genuinely clever, circuit-level tricks for shaving qubits and Toffoli gates. **This primer is a guided tour of those techniques**, explained from quantum-circuit first principles for a reader who has just learned the basics — covering all three of the challenge's objectives (minimize the spacetime product, minimize qubits alone, and map the clean (qubit, Toffoli) Pareto frontier; see §2).

2. What We Are Optimizing — The Score and Why Toffoli Gates Are Expensive

2.1. The score

Score objective (lower is better)

$$\text{score} = \text{peak\_qubits} \times \text{avg\_executed\_Toffoli}$$

This is the ecdsa.fail benchmark's cost function for a quantum circuit that computes an affine elliptic-curve point addition on secp256k1 (Bitcoin's curve). The two factors penalize two different resources:

- **Peak qubits**: the maximum number of qubits simultaneously in use at any instant. This is the "width" of the circuit, not the total ever allocated. On real hardware, idle qubits still occupy physical space on the quantum processor.

- **Average executed Toffoli**: the Toffoli (CCX) gate count, *weighted by how often each gate fires*, averaged over the 9024 test inputs the challenge uses. Not just the gate count, but gates × firing_rate.

2.2. Why Toffoli gates specifically?

On a **fault-tolerant** quantum computer (the kind needed to break Bitcoin-scale cryptography), gates fall into two classes:

- **Clifford gates** (CNOT, Hadamard H, Phase S, Pauli $X/Y/Z$): implemented efficiently using the surface error-correcting code at very low physical-qubit overhead.
- **Non-Clifford gates** (Toffoli/ T-gate): require **magic states** — specially prepared ancilla qubits that take a long time and many physical qubits to distill via "magic state distillation." Each Toffoli gate consumes one distilled magic state.

The rough conversion: 1 Toffoli = exactly 7 T-gates (Gosset et al. 2013, tight lower bound).

So in this regime, **Toffoli gates are like currency**: each one costs approximately the same fixed price (one magic state). The peak qubit count is like "desk space" — idle qubits still occupy factory floor. The product peak_qubits × Toffoli captures the spacetime volume of running the circuit.

Common metric confusion

This challenge uses `avg_executed_Toffoli`, NOT:

- T-count (= 7× Toffoli, a different metric)
- T-depth (Selinger 2013 — a depth trick with 4 ancilla per Toffoli; irrelevant here)

The distinction matters: average-executed rewards conditional execution (replay), emitted T-count does not.

2.3. The current SOTA

The best known result as of 2026-06-27 (BitWonka, commit `d44cad3`, trailmix_ludicrous family):

1152 qubits × 1,364,230 avg Toffoli = 1,571,592,960 score

2.4. Three objectives, not one

This is important context for the whole document. The ecdsa.fail challenge is really **three optimization tracks**, and a technique that is optimal for one can be irrelevant — or even counterproductive — for another:

1. **Minimize the spacetime volume (the Q×T score competition)** — the main subject of this primer. Minimize the *product* `peak_qubits` × `avg_executed_Toffoli`. Both resources matter, and every move is judged by the exchange rate (§10). The SOTA above (1152q × 1.36M) lives in this track.
2. **Minimize qubits regardless of Toffoli (the low-qubit competition)** — a question of academic curiosity: *how few qubits can a correct point-add possibly use?* Toffoli count is *unconstrained*. This track maps the qubit **floor**; the deepest technique here is EEA register sharing (§7.15), reaching 851q → 829q (far below anything score-competitive). Under this objective, spending 8× more Toffoli to save 10% of qubits is *free*, because Toffoli never enters the scoring.
3. **Explore the (Q, T) Pareto frontier** — especially the *useful corner* where **both** Q

and T are lower than the published resource estimates for a **single secp256k1 point addition**: **≤1175 qubits / ∼2.69M Toffoli** ($= 2^{21.36}$), the Babbush et al. (Google Quantum AI) space-optimized figure tabulated in Schrottenloher 2026 (arXiv:2606.02235, Table 1; original arXiv:2603.28846). *(Watch the units: the often-quoted ≤1200q / ≤90M Toffoli is the **full** ECDLP/Shor run — ∼28 windowed point additions plus overhead — whereas the scored circuit here is **one** point addition, so the per-point-addition figure is the correct reference.)* A point is Pareto-optimal if you cannot lower one of Q or T without raising the other. The goal here is not a single score but a *clean, forkable basis curve* at each qubit count (see the 1153q→1133q Pareto bases, §14.3). To stay reusable, these circuits **deliberately disable the most overfit lever — dead-CCX deletion** (§11), whose hard-coded gate-index lists are tied to one exact op stream. They are *not* fully value-exact, though: they still use the route's calibrated **approximations** (comparator-width narrowing, carry truncation, and at the lowest rungs active-width trimming — all island-exact, §11), so each still needs its own nonce hunt. The point is a *cleaner, more forkable* reference than a dead-CCX-stacked SOTA — not an all-inputs-provable one.

These are genuinely different games. Most of this primer optimizes track 1, but several techniques (register sharing, the clean Pareto bases) only make sense under tracks 2 and 3. Keep the three objectives separate in your head: many "is this worth it?" questions only have an answer once you know *which* track you are on.

3. The Reversibility Constraint — Why Quantum Circuits Are Hard to Optimize

3.1. You cannot delete information

Quantum circuits must be reversible. This is not a convention — it is required by quantum mechanics. Every gate in a quantum circuit must have an inverse (you can always "run it backwards"). This means:

- **You cannot overwrite a qubit.** If qubit A holds value x and you want to compute $f(x)$ into A, you cannot just replace it. You'd lose x forever, breaking reversibility.
- **You cannot erase a qubit.** If a qubit is entangled with the rest of the system (as intermediate results always are), setting it to $|0\rangle$ forcibly would violate unitarity.

The only way to "free" a qubit is to **uncompute** it — run the computation that created its value in reverse, returning it exactly to $|0\rangle$. Once it's $|0\rangle$, it can be freed.

3.2. Consequence: intermediate values cost qubits until uncomputed

Every time your circuit computes an intermediate result (a carry bit, a partial product, a comparison flag), it must allocate a fresh qubit. That qubit remains **live** — occupying a slot in the peak count — from the moment it's computed until the moment it's uncomputed.

Example: An n-bit addition $a + b$ computes n carry bits, one per bit position. Each carry qubit is live from when it's computed until the carry chain is reversed. If you compute the addition at step 100 and uncompute it at step 200, those n carry qubits are live across the entire steps 100–200, contributing to the peak at any instant in that range.

This is why almost every qubit-reduction technique boils down to: **make intermediate values die sooner** (so they aren't live at the peak) **or don't materialize them at all**.

3.3. Reversibility also doubles Toffoli counts

In a classical circuit, you compute X, use X, then discard X. In a reversible circuit, you compute X (costs Toffoli), use X, then *uncompute* X (costs the same Toffoli again). So every logical AND operation normally costs $2\times$ — once forward, once backward.

This is the central pain point that Measurement-Based Uncomputation (§9.1) solves.

4. The Challenge Circuit — Elliptic Curve Point Addition

4.1. What it computes

The circuit computes one **affine elliptic curve point addition**: given quantum register $P = (P_x, P_y)$ and classical constant $Q = (Q_x, Q_y)$, compute $P + Q = R$ in place, where the arithmetic is modulo the secp256k1 prime $p = 2^{256} - 2^{32} - 977$.

The mathematical operations, in order:

```
dx = Px - Qx              (mod p subtraction)
dy = Py - Qy
lambda = dy / dx          (modular inverse + multiply: the expensive GCD step)
new_Px = lambda^2 - Px - Qx
new_Py = lambda*(Px - new_Px) - Py
```

4.2. Why modular inverse is the bottleneck

Computing dy/dx mod p requires computing dx^{-1} mod p, the modular inverse of dx. There is no simple formula — it requires running the **Extended Euclidean Algorithm (EEA)** or equivalent, which takes ≈ 530 steps of binary GCD (divstep) operations. Each step involves:

- A comparison (swap decision: is $|u| > |v|$?)
- A conditional swap of two 256-bit registers
- A conditional subtraction of two 256-bit registers

This accounts for roughly 90% of the circuit's Toffoli count.

4.3. How an in-place modular inverse actually works: the dialog transcript

A classical EEA tracks two "Bézout coefficient" registers alongside the operands to build the inverse. Doing that reversibly would carry several extra 256-bit registers through all $\sim$530 steps — very expensive in qubits. The trick that makes the circuit affordable (Schrottenloher Algs 2–4, descended from the Khattar–Gidney "Dialog") is to **split the algorithm into a cheap construction pass and a cheap reconstruction pass that share a small recorded tape**:

- **Pass 1 — GCD construction (records the tape).** Drive the binary-GCD pair (u, v) from (p, dx) toward $(1, 0)$. The only data-dependent information at each step is the small **decision symbol** (subtracted?, swapped?, shift) — a few bits. Record those onto a **transcript tape**. This pass uses *non-modular* arithmetic with $\sim$n free ancilla, so it is cheap. It *consumes* dx into the tape.
- **Pass 2 — Bézout reconstruction (replays the tape).** Read the tape **in reverse**, maintaining one *modular* pair (r, s) and applying only linear updates **controlled solely by the recorded bits**: $s = 2s$ mod p; if subtracted: $s \mathrel{+}= r$; if swapped: swap(r, s). No comparisons, no operand-dependent branching — just replay.

The elegant part (why no explicit inverse is ever built). Because the replay updates are *linear*, the seed you feed into pass 2 comes straight out. Seeding $(r, s) = (1, 0)$ reconstructs dx^{-1}; seeding $(r, s) = (y, 0)$ instead yields $(0,\ y \cdot dx^{-1} \bmod p)$ **directly** — the inversion *and* the multiply-by-y happen together, with the inverse never materialized as its own register. And reading the tape **forward vs reversed flips between $y \cdot dx$ and $y \cdot dx^{-1}$** — the same machinery is a modular multiply or a modular divide by replay direction. This is how the SOTA does both EC slope steps with **one inversion engine and one tape** (§13).

Where the peak lives — and the one knob that sets the Pareto split. The qubit peak is *not* during construction (operands shrink, freeing room for the tape); it is during **reconstruction**, where the compressed transcript (∼2.355n) and the two n-bit modular registers r, s co-reside:

$$\text{peak} \approx \text{compressed transcript} + 2\,\text{modular registers} + O(\sqrt{n}) \approx 4.355n.$$

So *both* score factors are set by the reconstruction step — that is where transcript compression (§7.8) and the vented adders (§9.1) are aimed. And one clean knob sits behind the whole (Q, T) Pareto curve: spending **n extra ancilla during reconstruction** lets you use a cheap Gidney adder (2n Toffoli) instead of a CDKM adder (3n) — that single choice *is* the space-optimized-vs-gate-optimized split in both Babbush and Schrottenloher.

Jump-GCD (fewer steps). A plain binary GCD does one reduction per trailing-zero bit. A **jump (Stein-style) divstep** strips up to `jump=2` trailing zeros at once, so the algorithm finishes in fewer steps — directly fewer adder/comparator firings, hence fewer Toffoli. The cost is a slightly larger symbol alphabet to record (5 symbols instead of 3), which the codec absorbs. The SOTA runs `jump=2` (258 steps).

4.4. The key design choice: Q is classical

The second point Q is a **fixed constant** (the secp256k1 generator point, known at circuit compile time). The current SOTA keeps Q as classical bit-patterns (`BitIds`) rather than quantum registers, eliminating **512 qubits** from the peak — the single largest qubit reduction in the circuit's history.

4.5. Why secp256k1's prime is cheap: pseudo-Mersenne reduction (the math background)

Every arithmetic step works **modulo** $p = 2^{256} - 2^{32} - 977$. Doing $\bmod p$ naively means dividing by a 256-bit number — a full multiprecision division, expensive in any circuit. The whole reason this challenge is approachable is that p is a **pseudo-Mersenne (Solinas) prime** — it sits just below a power of two. Write

$$p = 2^{256} - c, \qquad c = 2^{32} + 977 \quad \text{(only 33 bits wide)}.$$

A quick word on the names (they describe a family of "reduction-friendly" primes):

- **Mersenne prime** — $p = 2^k - 1$. Reduction is *trivial*: $2^k \equiv 1$, so you fold the high bits straight onto the low bits with one add. (Too rare to land on a secure 256-bit curve, but the ideal everything else approximates.)
- **Pseudo-Mersenne prime** — $p = 2^k - c$ with c *small* (here 33 bits). Now $2^k \equiv c$, so the fold is "high bits $\times c$, then add." *Pseudo-Mersenne* emphasizes that c **is small**, which keeps the $H \cdot c$ multiply cheap.
- **Solinas prime** (a.k.a. *generalized Mersenne*, Jerome Solinas, 1999) — c is not just small but a **sparse signed sum of a few powers of two**. For secp256k1, $c = 2^{32} + 977 =$

$2^{32} + 2^{10} - 2^5 - 2^4 + 1$ (see NAF, §9.7) — only ∼5 signed terms. *Solinas* emphasizes that c **is sparse**, so the $H \cdot c$ "multiply" is really a handful of *shifted adds/subtracts*, not a general multiplication.

secp256k1's prime is **both at once**, and the two properties cut different costs: being pseudo-Mersenne (small c) means *one* fold suffices; being Solinas (sparse c) means that fold is a few shift-and-adds. Throughout this primer "the Solinas fold" and "pseudo-Mersenne reduction" refer to the same operation — folding the top bits down by $+c$ — and "Solinas constant" / "the fold constant" both mean c (called f in the code).

The one identity everything rests on is

$$2^{256} \equiv c \pmod{p}$$

(because $2^{256} - c = p \equiv 0$). In words: **anything at bit position 256 or higher is congruent, mod p, to the same thing multiplied by the tiny constant c and dropped back into the low bits.** So a wide value $H \cdot 2^{256} + L$ reduces as

$$H \cdot 2^{256} + L \;\equiv\; L + H \cdot c \pmod{p}.$$

You never divide by p; you **"fold" the overflow H back down** by multiplying it by the 33-bit c and adding — a couple of short operations instead of a division. This single fact is the source of the entire secp256k1-vs-generic-prime cost gap: in Schrottenloher's accounting the constant-adder is **15% of all Toffoli for secp256k1 but 34% for a generic prime**, and modular doubling **8% vs 24%** (paper Table 3; secp space-opt $2^{21.19}$ vs generic $2^{21.78}$).

The intuition to carry forward: a modular reduction is normally "subtract p until in range," but for a pseudo-Mersenne prime it becomes **"fold the top bits into the bottom bits,"** cheap precisely because c is small. §9.18 turns this into the actual reversible gadgets (mod-double, mod-add, the $+f$ fold); §9.7 (NAF) makes the fold constant itself cheaper.

5. Measuring Progress — The Peak Qubit Profile and Hot Spots

5.1. The live-qubit-vs-time profile

To reduce the peak, you first need to know **where** the peak occurs. Plot how many qubits are live at each moment in the circuit timeline:

```
qubits  |
live    |         ##########
        |    ####################
        |  ########################
        |################################
        +--+----+--+------+--+----+----> time
                   ^
                   peak = 1152 here (the "binder")
```

The highest point is the **peak**, and the phase executing at that height is the **binder**. Only operations alive at the binder instant matter for qubit reduction.

Key tool: `TRACE_PEAK=1 TRACE_PHASE_ACTIVE=1` in the build flags. This instruments every `alloc_qubit`/`free_qubit` call and prints the name of the phase executing when the maximum is hit.

5.2. The plateau problem

After shaving the single tallest binder, you often discover the peak hasn't moved — because several *independent* phases all reach the same height. This is a **co-binder plateau**:

```
qubits  |
live    |    #####   #####     ####
        |  ########################
        |  ########################  <- plateau at 1152 (Phase A, B, C all tie here)
        +----+--+----+---+--+---+---> time
```

The reported peak doesn't drop until **every** plateau member is individually pushed below the current height. This changes the strategy fundamentally: you must work in "plateau mode" — find a local qubit reduction for each co-binder, then combine them all.

5.3. Emitted vs average-executed Toffoli

- **Emitted**: the raw count of CCX gates in the circuit, regardless of whether they fire.
- **Average-executed**: each CCX gate's contribution weighted by Pr[control fires], averaged over all 9024 test inputs. A CCX that only fires 5% of the time contributes only 0.05.

This distinction is critical because **conditional replay** (§9.3) can dramatically reduce average-executed without changing emitted at all.

6. Qubit Reduction: The Core Loop

Before diving into individual techniques, understand the general loop:

1. **Measure** — get the live-qubit-vs-time trace; find the peak height and the binder phase.
2. **Inventory** — list every qubit alive at the binder instant.
3. **Classify** — label each qubit by category (table below).
4. **Brainstorm** — for each non-essential qubit at the binder, propose a shave.
5. **Cost** — compute the Toffoli overhead vs qubits saved. Compare to the break-even exchange rate (§10).
6. **Re-measure** — apply the cheapest safe shave. The peak moves to a new binder. Repeat.

6.1. Qubit classification at the binder

Class	What it holds	Shrinkable?
Essential data	Inputs/outputs the binder reads or writes	Rarely
Scratch	Temporary arithmetic results (carries, partial products)	Often
Transcript / log	GCD branch decisions recorded for reversibility	Often — compress the log
Passenger	Allocated before the peak, consumed after, *idle at binder*	Yes — relocate its endpoints
Alias-candidate	Restorable duplicate of a value already live elsewhere	Yes — borrow as dirty scratch
Truncatable	Bits provably zero or irrelevant to the final output	Yes

6.2. Binary vs graded failure modes

A critical distinction before applying any technique:

- **Graded-failure levers** (carry truncation, comparator narrowing): correctness degrades *continuously* with aggressiveness. A small fraction of inputs go wrong. You can iterate on the parameter and search for a nonce where all 9024 test inputs avoid the failures.
- **Binary-failure levers** (register live-range holes, transcript compression, structural reuse): correctness is all-or-nothing. Either the circuit is value-exact on all inputs, or it's wrong on essentially *all* inputs. There is no graded near-miss ladder to climb. You must get the implementation mathematically correct before searching for nonces.

The trap

Applying the "tweak-and-triage" loop to a binary-failure lever. A structural width cut that's not value-exact will produce all-dirty garbage on every input, not near-misses.

7. Qubit Reduction Techniques (1–19)

7.1. Live-Range Holes: Uncompute Early, Recompute Later

The headline move. If a qubit holds a value V that the peak instant does NOT need (it was computed before the peak and will be consumed after), free it before the peak and rebuild it after.

```
Timeline view:
Before:  ...[V computed]=====[V held live across peak]=====[V consumed]...
After:   ...[V computed][uncompute V]  (peak happens)  [recompute V][V consumed]...
                                        ^^^^^^^^^^^
                                   V is absent at peak
```

Mechanics: Emit the reverse of the gates that built V (returning its qubits to $|0\rangle$ and freeing them), run the peak-owning phase with that freed headroom, then re-emit the forward gates to restore V for its later consumer.

Cost: For a value that cost C_0 Toffoli to compute:

- Save k qubits at the peak
- Spend $2C_0$ extra Toffoli (one uncompute + one recompute)
- The trade wins if $2C_0 < k \times$ (avgT/peak_q) (the break-even rate)

Real examples:

- **1211→1193**: A GCD-apply scratch block was uncomputed during a shift sub-phase that didn't use it, then reacquired after. Saved 18 qubits.
- **1166→1165** (`TLM_PARK_ODD_U0`): `u[0]` is provably 1 (GCD odd-u invariant). Apply $X \to |0\rangle$ to zero it, `loan_zero_qubit` to return the lane, then `restore_known_one` (apply X again) to restore. Near-zero Toffoli cost, -1 qubit.

The keep-alive dual: Sometimes the uncompute+recompute round-trip creates a wider transient intermediate than just holding the value. In that case, *keep* the value live across the middle operation. Choose whichever is narrower: holding footprint vs recompute peak transient.

7.2. Passenger Relocation

A **passenger** is a qubit that was allocated early and consumed late, but does nothing at the peak — it just sits idle.

The cheaper fix: Don't uncompute and recompute. Just **allocate later** (right before first use) and **free earlier** (right after last use). If both endpoints fall outside the peak window, the passenger vanishes from the binder count at zero Toffoli cost.

Check passengers first before resorting to live-range holes — relocation is free.

7.3. Range-Bound a Register to Drop a Guard Bit

Some registers need an extra "guard bit" (sign or overflow indicator) only because an intermediate could transiently go negative or overflow. If you can prove the value always stays in a non-negative range by reordering operations, delete the guard bit entirely.

Example (benhuang Lever B): The fold quotient register had bit 33 (a sign guard) because Solinas reduction terms could make the partial sum dip below zero. Reorder to apply positives first. Now the sum is always in $[0, 2^{33})$. Allocate 33 bits instead of 34. **Net: -1 qubit at zero Toffoli cost.** This produced the 1221→1220 drop.

General rule: In $+A - B$ sequences, applying positives first and subtracting last keeps partial sums non-negative, eliminating sign-guard bits.

7.4. Truncation: Keep Only the Bits That Matter

Carry chains, high words of partial sums, and intermediate registers often hold more bits than actually affect the final result.

Per-step scheduling: In a 530-step loop (the GCD), a single constant truncation width must be set safe for the worst iteration. A per-step schedule width(step) lets each iteration be exactly as tight as it can tolerate — saving qubits only at the steps that are actually at the peak.

Two truncation flavors:

- **Structural truncation (density-neutral)**: prove from circuit analysis that the dropped bits are always zero on all inputs.
- **Empirical truncation (island-exact)**: verify the dropped bits are never 1 on the 9024 challenge inputs specifically. Requires re-hunting a nonce after each change.

7.5. Free-and-Recompute a Cheap Value (The 1153→1152 Breakthrough)

When a dead-qubit scan finds **no** unused qubits at the peak, don't conclude "the floor is structural." Ask instead:

Is any held value a cheap function of the other values already live at the peak?

If yes, **free** that qubit, let the peak phase use its slot, then **recompute** the value from the live qubits afterward for negligible cost.

The cy0 trick (`d44cad3`, BitWonka): The qubit `cy0` holds the carry-in at bit 0. No dead qubits found by scan. But because the secp256k1 prime constant $f[0] = 1$:

$$\texttt{cy0} = \texttt{control\_bit} \text{ AND } (\text{NOT } \texttt{final\_a0})$$

Both `control_bit` and `final_a0` are already live at the peak. So:

1. Uncompute `cy0` using 2 CNOT-equivalent gates (≈ 0 Toffoli)

2. Return the freed lane via `loan_zero_qubit`
3. After the peak phase: recompute `cy0` from `control_bit` and `final_a0` (2 gates)

Applied 40 times (once per binding fold iteration): 40×4 CNOTs = 0 Toffoli. Saves 1 qubit. The 1153→1152 drop.

Generalized lesson: After a dead-qubit scan fails, scan again for "whose value is a CHEAP FUNCTION of other live qubits?" The second question catches what the first misses.

7.6. Dynamic-Width Registers

In a GCD algorithm, operands naturally shrink as computation proceeds. After k steps of binary GCD, the operands u and v have lost at most k bits from their high ends. A naive circuit allocates full 256-bit registers for all 530 steps.

Dynamic-width registers: At each step i, determine the actual live bit-width and only allocate that many qubits. Free the high bits as they become known-zero.

Ablation results (from the Shrunken-PZ q980 track):

```
Source-level dynamic PZ (universal schedule):     1027 qubits
+ thin support-tuned schedule (per-step widths):   990 qubits  (-37q)
+ counter/rotation metadata trimming:              983 qubits   (-7q)
+ quotient cap:                                    980 qubits   (-3q)
```

Thin schedule (island-exact): An even tighter version samples actual GCD widths over many random inputs and builds per-step bounds tight to that distribution. This is island-exact — the thin schedule may be too narrow for inputs outside the sampled set. Requires nonce hunting.

7.7. Known-Constant Teardown Before the Peak

When the GCD terminates, certain registers are in known, predetermined states: $A = 0$, $B = 1$, `ca` $= p$, $q = 0$. These are **classical constants** — not quantum information.

What to do: XOR the known constant *out of* the register (zeroing it) before the peak computation, then XOR it *back in* afterward.

Example: Register `ca` holds $p = 2^{256} - 2^{32} - 977$ at termination. Before the lambda multiply: XOR p out of `ca`, returning it to $|0\rangle$. Free the register. Run the lambda multiply with the freed headroom. After the lambda multiply: XOR p back in for uncomputation.

Toffoli cost: XOR (CNOT) operations are Clifford gates — free. The teardown and recreation cost 0 Toffoli. This is a pure qubit win.

7.8. Transcript / Log Compression

The GCD must record its decisions. At each of the 530 divstep iterations, the algorithm makes data-dependent decisions: did we swap? did we subtract? These decisions are recorded in a **transcript** (also called "dialog tape") because they're needed to run the GCD backward during uncomputation. With one qubit per decision, the raw transcript is ≈ 772 bits long.

Compression techniques:

1. Base-5 codec (trailmix_ludicrous): Each 3-step window of (`subtracted`, `swap`, s_2) has 8 possible bit patterns, but only 5 are reachable. Encode each 3-step window in $\lceil \log_2(5^3) \rceil = 7$ bits instead of 9 raw bits.

```
Raw:        1 + 3*257 = 772 qubits
```

```
Compressed: 1 Step0 (2b) + 85 Triples (7b each) + 1 Pair tail (5b) + t1 flag (1b) = 603
    qubits
```

2. K5 exact 11-bit codec: Enumerate all reachable 5-step transcript windows exhaustively. If there are exactly $2^{11} = 2048$ reachable patterns, encode each in 11 bits. Information-theoretically optimal. Implemented via a borrowed-ancilla 17-CCX reversible codec. Converts 1203q→1193q.

3. Three-step tail codec (32 symbols): The last few GCD steps are heavily constrained. Enumerate the exact reachable set (32 symbols), encode in 5 bits. The 1185q→1170q drop.

4. Streaming decompression: Instead of decompressing the entire transcript before use, decompress one window at a time, use it, then immediately clear that window. Only one window is "open" at a time, reducing peak overhead from the codec.

The hard constraint: The codec must be a **bijection** — a perfect 1-to-1 map between reachable patterns and codes. Every codec must include a self-test proving bijectivity over the full reachable support.

7.9. Plateau Decomposition: Lower Every Co-Binder Together

When multiple independent phases all tie at the same peak height, you're in plateau mode. Lowering one phase alone doesn't change the global peak.

The protocol:

1. List ALL phases tied at the current peak (not just the first one). Build a "co-binder ledger."
2. Find a *local* qubit reduction for each binder independently.
3. Verify each local reduction doesn't accidentally raise a neighboring phase.
4. Combine all compatible local reductions in one package.
5. Re-measure the combined package.

Real example — the q1170 plateau (9-way co-bind, all tied):

Phase	Local fix	Effect
Solinas square	smaller square segment notch	square peak 1170→1169
apply final ripple	low-qubit final ripple variant	ripple 1170→1162
apply double/reverse halve	fold carry parking + per-step map	1170→1169
special overflow/underflow folds	fold parking + per-step map	1170→1169
non-final apply ripple	source-as-carry suffix for one lane	1170→1169

Each was independently developed, then combined. Only when all binders were locally reduced did the global peak drop to 1169.

7.10. Dynamic Headroom Clamp

Define a **circuit-wide qubit ceiling** and have every arithmetic primitive clamp its carry/chunk width to stay under it:

```rust
fn target_qubit_headroom(circ: &Circuit) -> usize {
    TLM_TARGET_Q - circ.active_qubits()
}

fn varchunk_adder(circ: &mut Circuit, bits: usize) -> ... {
    let k = min(scheduled_k, target_qubit_headroom(circ));
    // ... use k as the carry chunk width ...
}
```

Effect: No single call can exceed `TLM_TARGET_Q` total qubits. **Lower `TLM_TARGET_Q` by 1 → peak drops by 1**, as long as the narrower adder is still correct.

The tradeoff: Tighter ceilings force narrower carries → more carry-chunk splits → more Toffoli. The break-even exchange rate governs when to tighten.

Crucial nuance — the clamp is only a *ceiling*, not a lane-freeing lever. Lowering `TLM_TARGET_Q` declares an aspiration; it does nothing unless there are *actually freeable lanes* for the narrower arithmetic to fit into — "a clamp with no freed lanes just panics or fails to fit." So a clamp drop must be **paired with companion levers that genuinely eliminate live qubits** at the peak. The **1156 → 1153** drop is the textbook example — it stacked three coordinated levers:

1. **The clamp** itself: `TLM_TARGET_Q` 1156 → 1152 (the binding peak lands at 1153, one above the aspiration — the irreducible co-resident set).
2. **Zero-cin fold** (`TLM_FOLD_CHUNK_ZERO_CIN`): the fold-chunk loop's persistent carry-in ancilla `cin0` is *no longer allocated at all* — the first chunk's carry-in is provably 0, so a new "zero-cin boundary-erase" path (`fold_boundary_erase_zero_direct`) does the carry cleanup without it. This is the actual freed lane (a live-range elimination, cf. §7.11) that the clamp reclaims.
3. **FFG vent-count cap** (`TLM_FFG_MAX_G = 47`): caps the FFG half-product fold's clean-vent count g so fewer transient vent qubits co-reside at the peak (`capped_g` = min(`scheduled_g`, `cap`)).

And the qubit drop alone is not enough to *win*. Pushing the peak 1156 → 1153 makes the clamped adders run on tighter chunks → more carry work → avgT went *up* by +11,369 (to 1,392,603), i.e. ∼3,790 Toffoli/qubit, far above break-even. That "clean q1153 base" was **rejected** until Toffoli levers (the cinc cascade replacement §9.17 + iterated dead-CCX §9.8) were stacked back on top to carry it over the line — a perfect illustration of the product-race rule (§10): a qubit drop is only real on the score once enough Toffoli is recovered to pay for it.

7.11. Lazy Carry Liveness

A specific instance of live-range shortening for carry-in qubits. The carry-in qubit was being allocated before a chunk loop and held live across all N iterations, even though it was only needed at the loop boundaries.

Fix (`TLM_FOLD_CHUNK_LAZY_CIN0`):

```
Before: carry_in allocated -> live across all N chunk iterations -> used only at
    boundaries
After:  carry_in allocated inside boundary_erase() -> used -> freed inside boundary_erase
    ()
```

The carry-in qubit's liveness is reduced to just the boundary operation. Peak saved: 1 qubit per chunk at 0 Toffoli cost.

7.12. In-Place Decode: Hold Blocks Compressed

For persistent register blocks that live across the peak, encode them compressed and decode *in place* only at the single use site.

Before: Full-width raw K2-pair block (6 lanes) held live across the whole peak. **After**: 5 compressed cells + one zero lane stored at rest; decode in place only at the use site (allocate the 6 lanes transiently, use them, re-encode to 5 cells). During the rest of the circuit, only 5 cells are allocated. This is `DIALOG_GCD_K2_APPLY_INPLACE_RAW_BLOCK=1`, the 1218→1215 qubit drop.

7.13. Borrowed-Carry Fusion

Standard carry/borrow vectors allocate fresh qubits. But if an adjacent scratch register is currently known $|0\rangle$, borrow its qubits as the carry vector instead:

```
borrowed_const_fold_carries(circ, need, borrowed_carries);
// Uses borrowed[..need] as carry vector; no fresh allocation
// At exit: borrowed restored to |0>
```

Net effect: No fresh qubits allocated for the carry prefix. The peak doesn't rise.

Hard requirement: The borrowed qubits must be genuinely $|0\rangle$ at the borrow point and restored to $|0\rangle$ before the borrower reads them again as data. Getting this wrong causes silent data corruption. (`DIALOG_GCD_SPECIAL_FOLD_BORROW_CARRIES=1`.)

7.14. Passenger Ghosting via HMR

Standard passenger relocation: Free before peak; recompute after. Requires the value to be recomputable from live data (costs Toffoli).

HMR ghost (more aggressive): Measure the passenger qubit in the Hadamard (X-basis). The qubit is freed. The measurement result (a classical bit m) is recorded. Later, reconstruct the value algebraically and apply a classically-conditioned Z correction to fix the phase.

Why this works: An X-basis measurement of $|\psi\rangle$ projects it into $|+\rangle$ or $|-\rangle$. The measurement outcome m tells you which. If $m = 1$ (got $|-\rangle$), a phase (-1) was applied to the computational-basis components — the CZ correction cancels this exactly. After the correction, the measured qubit is factored out and can be freed without information loss.

In the q980 EC point addition:

- dy (256-bit) computed before the GCD inversion. After GCD, λ and dx are live.
- The identity $dy = \lambda \cdot dx$ holds by construction.
- HMR-ghost dy before the GCD peak (measure it, record m).
- At the end: reconstruct $dy = \lambda \cdot dx$, apply phase correction via m.
- Freed 256 qubits across the peak at the cost of one modular multiply for reconstruction.

7.15. EEA Register Sharing — The Low-Qubit Frontier

The deepest qubit-reduction idea, and the headline technique for the low-qubit competition (§2.4, track 2 — minimize peak qubits, Toffoli unconstrained). This is what drives the lowest qubit counts ever recorded on the challenge: the 948q → 851q → 829q descent. Because that competition does not score Toffoli at all, register sharing is free to spend gates lavishly in exchange for lanes — exactly the trade it makes.

The naive EEA register budget. The extended Euclidean algorithm carries four full-width 256-bit registers through all ∼530 steps:

- A, B — the two operands (the shrinking u, v)
- CA, CB — their two Bézout cofactors (the running a, b with $a \cdot dx + b \cdot p = \gcd$)

Naively that is $4 \times 256 = 1024$ qubits just for the inversion core, before any scratch.

The sharing observation (Proos–Zalka 2003, the origin of "inversion dominates point-add space"): the operands and their cofactors are *complementary in size*. As the algorithm runs, operand A loses high bits (it shrinks toward 1) while its cofactor CA grows from zero. At every step the sum $\text{bitlen}(A) + \text{bitlen}(CA)$ is bounded by roughly one word. So A and CA can be **packed into a single physical 256-bit register**: A in the high part, CA in the low part, with a small *bit-length tracker* recording where the boundary sits. As A shrinks and CA grows, the boundary simply slides — but the total never exceeds one word.

```
Naive:   [====== A ======][== CA (mostly zero) ==]   two separate 256-bit words
         [====== B ======][== CB (mostly zero) ==]

Shared:  [== A ==|== CA ==]   ONE word; boundary slides right as A shrinks, CA grows
         [== B ==|== CB ==]   ONE word; a 9-bit length tracker remembers the split
```

Four full registers collapse to **two "work words" + a handful of small length/control trackers**. This is the structural source of Luo et al. 2026's compact exact reversible inversion at $3n + 4\lfloor \log_2 n \rfloor$ qubits (1333q for $n = 256$ as a standalone inversion). In the Shrunken-PZ track, packing $A \| CA$ and $B \| CB$ plus dirty-catalytic borrowing and gate-hosting drove the inversion core down to `REGISTER_SHARED_PAPER_INVERSION_QUBITS = 3*256 + 52 = 820` qubits, giving an **851-qubit peak** for the whole point-add — a 97-qubit drop below the prior 948q record, and the 851→829q race then squeezed out another 22.

Where the Toffoli goes (and why that's fine here). Packing is not free in gates: a shared word has a *data-dependent boundary* — "where does A end and CA begin?" depends on values computed at runtime. So every arithmetic step must first **locate and realign that boundary with a full-width comparator**, and because the circuit is reversible, that comparator is recomputed *before and after* each hosted gate:

```
Cost to realize ONE useful Toffoli on a packed word:
   toggle_gate ; ccx ; toggle_gate
   where toggle_gate ~= a full-width comparator ~= 12 x width ~= 3,075 Toffoli at width
    256
```

Across 530 divsteps this lands the 851q route at ∼464.5M average Toffoli (vs ∼54.8M for the unshared 948q route). **Under the low-qubit objective this is a non-issue** — Toffoli does not enter the score, so spending ∼8× more of it to buy 97 qubits is precisely the intended trade. (For contrast, under the $Q \times T$ *product* objective the very same packing would be ∼250× over break-even, so this technique belongs squarely to the low-qubit track, not the score track. The two objectives have genuinely different optimal constructions — see §10 for the exchange-rate math behind that statement.)

How to read the 851q figure

851q is a carefully-scoped **lower-bound witness** — a research milestone that says "this width looks reachable," with its modeling assumptions documented openly in the source. It is exactly the kind of result objective 2 (§2.4) is meant to surface. Three things are worth knowing so you read the number for what it is:

- **The deepest figure rests on a *classical* feasibility model.** `register_shared_eea.rs` computes on `U256`/`U512` integers — a classical simulation rather than an emitted gate stream. Its job is to confirm the packing invariant $l_t + 1 + l_q + \text{bitlen}(r) \leq n + 3$ holds at all 1,479 steps — i.e. that operand and cofactor genuinely *fit in one word*, establishing the packing is feasible in principle. The authors flag this transparently: *"an analysis oracle ... passing this oracle does not establish a challenge-valid qubit count"* (i.e. a feasibility model, to be paired with a hardened emission before it counts as a finalized circuit).
- **The lowest counts use gate-hosting whose zero-claims are *sampled* rather than proven** (see §7.16). The route is honest about which guarantees are still open: `q945_local_hosts.rs` notes the reachable-state zero claims are not yet certified and lists the remaining "Q946 hardening" prerequisites; `q944_gate_host_lifecycle.rs` verifies the hosting exhaustively *at small widths* and extrapolates to 256 bits.
- **The status is labelled, not hidden.** `Q949_ROBUST_ENVELOPE_FRESH_VALIDITY_CLAIMED = false` and the `proof_status` fields mark these as lower-bound explorations rather than finalized circuits — good hygiene for a frontier-mapping result.

In short: the 0/0/0 run check confirms the **outputs are correct on the hunted island**, and the 851/829q figures show **how low the width can plausibly go**. The lowest count backed by a fully-emitted, island-validated circuit with *no* outstanding hosting assumptions is the **948q** route — the natural "validated" milestone to cite — while 851q/829q are the further-reaching witnesses on top of it. Either way, the durable, fully-sound takeaway is the structural mechanism: operand/cofactor packing.

The lesson. Register sharing is *the* route below ~948 qubits, and the Proos–Zalka / Luo-2026 mechanism is the structural source worth knowing. It is the right tool when your objective is pure qubit count. The broader point generalizes both ways: **know which competition you are playing before pricing a lever** — a move that is a clear win for the low-qubit objective (trade unlimited Toffoli for lanes) can be a clear loss for the Q×T objective, and vice versa.

7.16. Gate-Hosting: Borrowing an Idle Lane Instead of Allocating

A workhorse peak-reduction technique — it powers the 948 → 851 drop (§7.15), and it doubles as a clean illustration of the value-exact vs island-exact distinction at the level of a *single borrowed qubit*.

The idea. Every reversible gadget needs **scratch ancilla** (a multi-controlled-X needs a lane to accumulate the AND of its controls; a carry chain needs carry bits; a comparator needs a borrow bit). Allocating a fresh $|0\rangle$ for each is what drives the peak up. But at the instant you need scratch, some lane *elsewhere* is often idle — the high bits of a shrunk number, a register between its last read and next write, an idle metadata lane. **Gate-hosting borrows that idle lane as scratch, then restores it before its owner needs it again** — so the ancilla costs no new peak qubit. The scratch is "hosted" inside a register that belongs to something else (the codebase has explicit per-step tables naming which `A/B/CA/CB/Q` lane+bit hosts each gate). This generalizes §7.13 (borrowed-carry fusion) to *any* gate's scratch.

The hot-desking intuition. Qubits are desks in a crowded office. Allocating a fresh ancilla = renting a new desk (the office gets more crowded). Gate-hosting = doing your hour of work at a colleague's empty desk while they are at lunch, then clearing it spotless before they return — no new desk, the peak never rose.

Two flavors:

- **Clean hosting** — the borrowed lane is known $|0\rangle$. Use it directly, restore it to $|0\rangle$.
- **Dirty hosting** — the borrowed lane holds unknown live data ψ. You can still borrow it via a trick that disturbs ψ and then exactly un-disturbs it (the Barenco "surrounded" pattern):

```
ccx(c0, c1, psi)        <- scribble onto the borrowed lane
ccx(psi, c2, target)    <- do the real work using it
ccx(c0, c1, psi)        <- un-scribble: psi restored to its original value
ccx(psi, c2, target)
```

  The two extra gates restore ψ perfectly. Cost: 4 Toffoli instead of 3 — a textbook -1 **qubit,** $+1$ **Toffoli** trade (great when the objective does not score Toffoli).

The principled, scaled-up form of dirty hosting is **conditionally-clean ancillae** (Khattar–Gidney 2024, "Laddered Toggle Detection"): a method to borrow dirty bits *as if they were clean*, with no allocation and no separate toggle-detection pass. It is what lets a multi-controlled-X run in $\sim$3n Toffoli with only $\log^* n$ truly-clean ancilla, and it underlies the cheap MCX/`cinc` primitives (§9.17). When you need a lot of scratch and have only dirty lanes, this is the formal tool.

It is a compile-time bet, not a runtime scan. A quantum circuit is static — you cannot peek at a qubit mid-run to check "is this $|0\rangle$?" (measuring would collapse it). So nothing searches for zeros at runtime. The hosting choices are **frozen into the gate list at build time**: the compiler asserts *"at step 437, lane 12 of register B will be $|0\rangle$, so host the scratch there,"* and the circuit runs that exact plan on every input. The whole question is whether that compile-time claim was *true*.

Proven-zero vs sampled-zero (value-exact vs island-exact, one qubit at a time). The zero-claims come in two grades:

- **Proven zero (sound, density-neutral):** the lane is zero for a *structural* reason — e.g. the high bits of an operand the GCD has provably shrunk, or a known-constant register. Safe for all inputs.
- **Sampled zero (island-exact, risky):** the claim "this lane is $|0\rangle$ here" is established by an **empirical census** — run many sample inputs, observe the lane was always zero, *assume* it always is. This is the grade the aggressive 851q borrows rely on. "The desk was empty every time I checked" is not "the desk is always empty."

How it breaks. If an input arrives (one not in the sample) that makes a sampled-zero host lane *non-zero* at borrow time, the hosted gate scribbles onto live data and the circuit silently returns a wrong answer — reversibility offers no safety net. The reason the shipped circuit still scores 0/0/0 is that the **nonce/test island is hunted to dodge exactly those inputs**: the 9024 graded inputs are chosen so none trips a bad borrow. So it never breaks on its own test set — because the test set was fitted to the circuit, not because the circuit is universally correct.

Takeaway. Gate-hosting is a perfectly sound, standard peak-reduction technique **when the host lane's state is proven**; it becomes an island-overfit assumption the moment the state is only *sampled*. The same value-exact vs island-exact split from §11 applies here at the granularity of a single borrowed $|0\rangle$.

7.17. Reset-Bounded Qubit-ID Compaction (Register Allocation for Qubits)

A free, value-exact post-pass that closes the gap between the *true* peak and the *scored* peak. The benchmark charges **the largest qubit *id* that appears in the circuit**,

not the true count of simultaneously-live qubits. The builder reuses freed lanes, but if its numbering is even slightly loose — a temporary gets a high id when a lower one was already free — the *max id* can sit above the real concurrent peak. That slack is pure wasted score.

The fix is classic register allocation, applied to qubit ids. After the whole op stream is built, run a compaction pass (`compact.rs`, introduced in the 1133 Pareto push):

1. Cut the timeline at every **unconditional reset** (`R`) or measurement (`Hmr`) — these destroy a lane's value, freeing its id for reuse from that point.
2. Build the **lifetime interval** of every non-IO temporary lane (first use → last use within a reset-bounded segment).
3. **Interval-color** those intervals onto the fewest ids (a min-heap of free colors keyed by lifetime end — textbook interval-graph coloring), while **pinning the four IO registers** so the evaluator still finds its inputs/outputs.

The result is the same circuit (every gate identical) renumbered so `max id` equals the coloring optimum. It is **value-exact and density-neutral** — it changes only the *labels* on wires, never the computation — so it stacks freely and needs no nonce re-hunt.

Why this is the right mental model. A qubit id is exactly a *register* in a compiler: lanes never live at the same time can share one id, just as non-overlapping variables share a machine register. Lifetime-coloring is how compilers minimize register pressure, and it is the provably optimal way to minimize `max id` for a fixed gate schedule. Whenever a circuit's reported qubit count exceeds its measured concurrent peak (check with `TRACE_PEAK`), this pass recovers the difference for free. (In the 1133 push the lever is present but gated off — 1133 came from active-width trimming, not compaction — so it is an untapped drop-in for any future rung whose `max id` exceeds its true peak.)

7.18. Windowed (Chunked) Materialization — the `F_CUT` Dial

The biggest peak lever of the 1693→1411 era. Many operations need a *materialized* operand — a full-width quantum register built from something cheaper. The classic case: to add the quantum-controlled value $f = \texttt{ctrl} \cdot a$ into an accumulator (where a is 256 bits), the naive circuit first builds all 256 bits of f, then runs a 256-bit ripple adder. Two wide things are now live at once — the materialized f *and* the adder's carry lane — and their sum is the peak.

The fix: never materialize the whole operand at once. Split the add into chunks. For each chunk: materialize only that slice of f, run a small adder over just that slice, then **immediately clear the slice with a measurement (HMR)** before moving to the next chunk. Only one chunk's worth of f is ever live, and the boundary carry between chunks is cleaned up with a short truncated comparator against the source bits.

```
Naive:   build all 256 bits of f=ctrl.a  ->  256-bit ripple add   (f + carry both wide ->
    tall peak)
Chunked: per block:  build slice of f  ->  add slice  ->  HMR-clear slice  ->  fix
    boundary carry
         (only one slice of f live at a time -> much shorter peak)
```

Why it's a *dial*, not a one-shot. The chunk boundary (`F_CUT`, `APPLY_CHUNKED_F_BLOCKS`) is tunable: each +1 narrows the materialized slice and lowers the peak by ∼1–2 qubits, at the cost of one more boundary comparator (a few Toffoli). So you "sink" the apply phase to the next floor by turning the dial, then re-tune it after every *other* lever lands. This single knob, re-cut at nearly every step from 1572q down to 1411q, did most of the era's peak work. It is the materialized-operand version of venting and per-step truncation: **process a wide**

value a window at a time so its full width never co-resides at the peak.

7.19. Shift-Free Scaling via Modular Doublings

A subtle qubit trap hides in "multiply by a power of two." Computing $x \cdot 2^k \bmod p$ the obvious way — `shift_left`$(k) \to$ `op` $\to$ `shift_right`(k) — needs a **spill register** to catch the k bits that fall off the top during the shift, plus overflow/sign flags. In the Solinas 2^{32} fold of the square tail, that parked ∼24 persistent flag qubits and pinned the peak.

The fix uses a number-theoretic identity: modulo p, multiplying by 2 *is* a modular doubling (`mod_double`, §9.18) — a shift that immediately folds its overflow back via $+f$, leaving nothing to spill. So $x \cdot 2^k \bmod p$ can be done as k **successive modular doublings** (and $\div 2^k$ as k modular halvings), value-identical to the shift route but allocating **no spill register and no flags**. It trades a handful of extra Toffoli for ∼24 freed qubits (`KARA_SOL_SHIFT22_DOUBLES`, the 1558q drop). The lesson: a "free" classical operation (a bit-shift) can be quietly *expensive* in a reversible circuit because the shifted-out bits must be stored, not discarded — and re-expressing it in the modular arithmetic you already have can dodge that storage entirely.

8. Toffoli Reduction: The Core Loop

1. **Measure** — get the Toffoli count; find the **hot-spot**. Note emitted vs average-executed.
2. **Classify** — identify what *kind* of Toffoli these are (table below).
3. **Pick the matching move** — the kind determines the technique.
4. **Cost it** — depth-neutral? qubit cost? worth it at the exchange rate?
5. **Re-measure** — the hot-spot migrates; repeat.

Classifying Toffoli by kind:

Kind of Toffoli at the hot-spot	The matching move
Carry-uncompute (reverse carry chain returning scratch to $\|0\rangle$)	Vent via MBU (§9.1)
Self-uncomputing scratch (comparators built then cleanly reversed)	Conditional replay (§9.3)
Data-controlled arithmetic (`cadd`, fold-sub on live data)	Fusion / avoid materialization
Majority gadgets (3-CCX majority in carry logic)	Ancilla-free majority (§9.12)
Redundant final cleanup	Skip it (§9.14)
Full intermediate products / wide rows	Avoid materialization (§9.5)
Adjacent algebraic stages (add/negate/subtract chains)	Algebraic fusion (§9.4)

9. Toffoli Reduction Techniques (1–19)

9.1. Measurement-Based Uncomputation (MBU)

The Toffoli lifecycle problem: An AND gate costs 4 T-gates (1 Toffoli) to compute. The uncomputation normally costs another 4 T-gates. Computing + uncomputing one AND = 2

Toffoli.

MBU (Jones 2013, applied to adder carry chains by Gidney 2018):

```
1. Forward: ccx(a, b, out)       -> 4 T-gates (1 Toffoli)
2. Use 'out' in downstream logic
3. Uncompute: measure 'out' in X-basis (Hadamard, then Z-basis measure)
   Result is classical bit m in {0, 1}
   - If m = 0: done, free 'out'  -> 0 Toffoli
   - If m = 1: apply CZ(a, b)    -> 0 Toffoli (CZ is a Clifford gate)
Total uncompute cost: 0 Toffoli
```

Why does this work? When you compute `out = a AND b` into a clean $|0\rangle$ ancilla, the system is in state $|a, b, a \cdot b\rangle$. Measuring `out` in the X-basis either succeeds cleanly ($m = 0$) or applies a phase kick (-1) to the $|a = 1, b = 1\rangle$ component ($m = 1$). The CZ gate corrects exactly that phase. After correction, `out` is disentangled from (a, b) and can be freed.

The key insight: **measurement collapses entanglement for free**. Coherent uncomputation must undo entanglement gate-by-gate; measurement does it in one shot.

The tradeoff: The `out` qubit must remain live from when it's computed to when it's measured (typically between the forward and reverse carry chains). This costs +1 peak qubit during that window.

Practical effect on adders: An n-bit ripple-carry adder normally costs $\approx 2n$ Toffoli (n forward, n reverse). With MBU on every carry-uncompute:

$$\text{Total adder cost} = n \text{ Toffoli (forward only)} + 0 \text{ (measured uncompute)} = n \text{ Toffoli}$$

This halves the adder cost — the most important constant-factor improvement in the SOTA.

9.2. The Vent Dial

§9.1 showed that MBU can *uncompute* one carry bit for **0 Toffoli** (measure it, then a free Clifford fix-up) instead of the 1 Toffoli a coherent uncompute would cost — the catch being that the carry qubit has to stay alive a little longer. Applying that choice to *one* carry in an adder is called a **vent**. The whole "dial" is just: *how many of an adder's carries do you choose to vent?*

Where the $3n - 2$ comes from. A reversible n-bit adder builds a chain of carry bits climbing up the word (the *forward* pass), uses the final carry, then unwinds that chain (the *reverse* pass) to return all the scratch qubits to $|0\rangle$ (this compute–use–uncompute cycle is the reversibility tax from §3). Tallying the Toffoli in that cycle for the hybrid carry adder used here gives a fixed baseline of about $3n - 2$ Toffoli when *nothing* is vented. Each carry you vent deletes one Toffoli from the reverse (uncompute) half. So with k vents:

$$\begin{aligned} \text{Toffoli cost} &= 3n - 2 - k && \text{(one fewer Toffoli per vented carry)} \\ \text{Peak qubits} &= \text{baseline} + k && \text{(but each vented carry stays live a bit longer)} \end{aligned}$$

The dial is therefore perfectly linear and perfectly priced: **each vent = −1 Toffoli, +1 temporary peak qubit.**

Why the qubit is only *temporary* — the "forward↔reverse window." A vented carry can't be freed the instant it's measured: its measurement outcome is still needed later, when the reverse pass reaches that bit position to clean it up. So the qubit is occupied only from when that carry is first computed (forward) until it is finally cleaned up (reverse) — the stretch in between is the "forward↔reverse window." Outside that window the lane is free, which is exactly why the $+k$ qubits don't add to the circuit's permanent footprint.

When to vent — it depends on *where* the adder sits in the timeline. Recall from §5 that the circuit's qubit peak happens during a few specific phases (the *binder*), while other phases have **slack** — spare qubits sitting idle. So:

- **Off-peak adders** (running during a phase with slack): holding +1 qubit there does *not* raise the *global* peak, so venting is **pure profit** — free Toffoli savings at zero score cost. Vent every carry you can.
- **On-peak adders** (running at the binder): every extra qubit pushes the peak — and the score — up. Vent only if the Toffoli saved is worth more than the qubit, judged by the exchange rate (§10).

How the SOTA automates it. `trailmix_ludicrous` doesn't choose by hand. It fixes a circuit-wide qubit ceiling `TLM_TARGET_Q`, and for each adder call vents exactly `TLM_TARGET_Q` − `active_qubits` carries — i.e. it spends *all* the spare headroom beneath the ceiling on Toffoli savings, and not one qubit more. An adder already at the ceiling gets 0 vents (no room); an adder with lots of slack gets vented heavily. This is the dynamic-headroom clamp (§7.10) and the vent dial working together.

Gidney 2025 streaming vented adder (open)

A newer construction (arXiv:2507.23079) that uses 2 clean + $(n-2)$ *dirty* (borrowed, §7.16) ancilla for $\approx 3n$ Toffoli. Partially implemented in `venting.rs` but **currently leaks phase** — the phase correction for the dirty ancilla is incomplete — so it is not yet in the SOTA. Expected $\approx n/4$ Toffoli savings per adder once fixed.

9.3. Conditional / Measured Replay (Average-Executed Only)

The idea: If a comparator flag is $|0\rangle$ on most inputs, measure the flag instead of keeping it coherent. Then replay the dependent computation conditionally on the measurement outcome.

```
1. Compute flag F into scratch ancilla (costs C Toffoli)
2. Measure F in X-basis -> classical bit m
3. If m = 0: dependent block is a no-op on this shot  -> 0 Toffoli from the block
4. If m = 1: replay the block classically conditioned on m  -> full cost
Average cost = C + Pr[F=1] * (block cost)
```

When F fires on fraction f of inputs: average-executed drops from (block cost) to $f \times$ (block cost). If $f = 0.05$, you pay only 5% on average.

The hard constraint: This ONLY works on **self-uncomputing scratch** — a comparator you compute, read under the condition, and cleanly reverse. It does NOT work on data-controlled operations or predicates reused downstream.

9.4. Algebraic Fusion: Collapse Add/Negate Chains

Adjacent modular-arithmetic stages that are algebraically equivalent to a simpler single operation can be collapsed to eliminate intermediate carry chains.

Example (`DIALOG_FUSE_C_FORM`):

```
Old:
  tx += 2*Qx    (mod add)
  tx = -tx      (mod negate)
  tx -= Qx      (mod sub)
  tx = -tx      (mod negate)
```

```
Algebraically (two negations cancel, constants combine):
  tx += 3*Qx    (one mod add)
```

Example (`DIALOG_FUSE_X_RESTORE`):

```
Old:
  tx = -tx      (mod negate)
  tx += Qx      (mod add)

Algebraically:
  tx = Qx - tx  (one operation)
```

Each removed negate/add/subtract eliminates a carry chain, a modular reduction fold, and a cleanup phase — saving significant Toffoli.

9.5. Witness-First / Deferred-Output Dataflow

Sometimes a cancellation only needs a *witness relation*, not the final output value. Computing the output early and then cleaning it is wasteful when the witness is cheaper.

The 4352cfb Toffoli cut (−10.8M Toffoli at fixed 980 qubits):

Key identity (from curve equation): `new_dy` $= \lambda \cdot$ `new_dx`. So to cancel λ, we do not need to compute `new_y` first. Instead:

1. Erase `dy` using: $dy = \lambda \cdot dx$ (subtract $\lambda \cdot dx$ from $dy \to$ zero)
2. Reuse the zeroed `dy` register as scratch
3. Compute `new_x`
4. Compute `new_dx` $= ox -$ `new_x`
5. Compute `new_dy` $= \lambda \cdot$ `new_dx` (the witness, not the output)
6. Cancel λ using `new_dy`/`new_dx`
7. Materialize `new_y` $=$ `new_dy` $- oy$ (only NOW compute the output)

Ablation:

```
zero-dy disabled, defer-y disabled:   110,227,695 emitted ops
defer-y materialization only:         105,573,887 emitted ops  (-4.6M)
both zero-dy + defer-y:                92,105,748 emitted ops  (-18.1M total)
```

General principle: Before materializing any output, ask: "does the next operation NEED the output, or just a function of it that's cheaper to compute directly?"

9.6. Karatsuba Modular Square (−22.4 Million Toffoli)

The largest single Toffoli reduction in the project's history.

The schoolbook problem: Computing λ^2 for a 256-bit number using schoolbook multiplication costs $O(n^2)$ multiplications.

Karatsuba: For a 256-bit number split as $\lambda = \text{hi} \cdot 2^{128} + \text{lo}$:

$$\lambda^2 = \text{hi}^2 \cdot 2^{256} + \Big[(\text{lo} + \text{hi})^2 - \text{lo}^2 - \text{hi}^2\Big] \cdot 2^{128} + \text{lo}^2$$

Instead of computing 4 products, compute only 3 squares: lo^2, hi^2, $(\text{lo} + \text{hi})^2$. Reconstruct the cross-term via additions (cheap Clifford operations).

Why −22.4M Toffoli: The squaring operation runs at EVERY GCD step (258 steps × 2 passes). Any $O(n^2) \to O(n^{1.585})$ improvement compounds massively across all those calls.

Density-neutral: The Karatsuba identity is a pure algebraic identity, correct for all integers.

Recursive Karatsuba: Tested on 128-bit sub-squares → net Toffoli LOSS (the overhead of the additions outweighs the sub-square savings at that level). Dead end.

The space/Toffoli tension (it cuts both ways). Karatsuba saves Toffoli but *needs scratch* for the three sub-products and the $(lo+hi)^2$ term. When the square sits at the qubit peak, the **opposite** move can win: drop back to a plain **schoolbook** square, which needs no middle-term scratch (fewer qubits) at the cost of more Toffoli. The SOTA toggles this per context (`ROUND84` "schoolbook x-tail") — Karatsuba where Toffoli is binding, schoolbook where *qubits* are. Same exchange-rate decision as everywhere else (§10): which axis is binding decides which square you want.

9.7. NAF Recoding of Constants

Non-Adjacent Form (NAF) uses signed digits $\{-1, 0, +1\}$ to represent constants with fewer non-zero digits:

$$977 = 2^{10} - 2^5 - 2^4 + 1 \quad \text{(4 signed terms vs 6 binary terms)}$$

In NAF, no two adjacent digits are both non-zero, and the number of non-zero digits is minimal. For 977: 4 terms instead of 6, saving ≈ 33% of the Solinas fold cost.

Density-neutral: The identity $977 = 1024 - 32 - 16 + 1$ is a mathematical fact, correct for all inputs.

9.8. Dead-CCX Elimination (Empirical Drop vs Structural Skip)

Some CCX gates contribute nothing to the answer — either their control is always zero, or their net effect cancels — yet the scorer still *charges* them. Deleting such "dead" gates lowers the average-executed Toffoli. There are two ways to find them, with **very different correctness profiles**, and the difference is the clearest example of the overfitting trap in the whole project.

(1) Empirical dead-CCX drop (the island-overfit lever). Run the circuit over a large sample of inputs (∼9 million reachable EC points, or many Fiat–Shamir test sets); record which CCX gates **never flip their target** — i.e. are *inert-but-charged* on the sample. Collect their positions into a `.idx` file (e.g. 13,873 op indices) and filter them out:

```
ops = ops.filter(|(i, _)| !drop_set.contains(i))   // delete the inert-but-charged CCX
```

This was the engine behind the SOTA average-Toffoli for a long time (a single drop was worth ∼13,000+ avg-Toffoli). **But it is distribution-exact / island-overfit, and dangerous for three reasons:**

- **It deletes gates that *could* fire on unsampled inputs.** The result is "clean on its island" (0/0/0 over the hunted 9024) but **not** provably correct off-island — a gate inert on every sampled input may be live on one you never tried, and the deletion then silently corrupts the output.
- **The indices are absolute positions in one exact op stream.** *Any* structural change — a clamp, a knob, a fold toggle, even reordering — shifts every later gate onto the wrong index, so a stale list deletes *live* gates (9024/141/141 all-shots breakage). Every variant must ship its own freshly-screened list and freshly-hunted nonce.

- **It does not improve the construction.** A genuinely better circuit would not *emit* these gates; the drop just hides them from the scorer, and it is the first thing invalidated by any real structural improvement. Treat it as a sophisticated nonce grind — worth knowing it exists (it is why the SOTA avg-T looked low), low-value to reimplement except to squeeze a *frozen* final artifact.

Iterated (two-pass) empirical drop (`DROP_DEAD_ROBUST_SECOND`): the drop is **iterable**. After the first drop reshapes the stream, re-screen the *already-dropped* stream — a second screen finds gates newly (or only now detectably) dead — drop a second, smaller list, and re-hunt the nonce with *both* drops on. Subset-monotone with diminishing returns (2nd pass $\sim$−2,411 avgT at 1153q). All overfitting caveats above apply at every pass.

(2) Structural dead-CCX skip (the sound, density-neutral lever — CURRENT SOTA). Instead of *sampling*, **prove** a CCX never fires on *any* input from circuit structure alone (known-constant carries, exact-remainder conditions, call-site bit patterns). Named predicates in `trailmix_ludicrous`: `TLM_FFG_SKIP_STRUCTURAL_DEAD_*`, `TLM_CUCCARO_SKIP_STRUCTURAL_DEAD_*`, `TLM_COMPARE_SKIP_STRUCTURAL_DEAD_*`. Keyed by call-site and bit-index, not sampled data, so they are **density-neutral** (correct on all inputs, stackable freely, no `.idx`, no re-screen).

The 6dafa07 pivot. The SOTA *replaced* the empirical drop entirely with structural skipping — which turned out to be **both sound and lower-Toffoli**. The general lesson in miniature: when an island-exact lever is doing real work, look for the *structural* reason those gates are dead and skip them provably. If the gates truly never fire, you can usually prove it — and the overfitting risk then vanishes for free. (See §11 for the value-exact vs island-exact framing.)

9.9. Classical Constant Folding

When one operand of an arithmetic operation is a **classical constant** (known at compile time), the operation can often be implemented with 0 Toffoli gates.

The mechanism: A `BitId` register holds a constant pattern as compile-time information. Operations with one classical operand:

- `qubit XOR 0` = identity (no gate)
- `qubit XOR 1` = Pauli X (free Clifford gate)
- `qubit + classical_constant mod p` = a series of CNOT-like operations — 0 Toffoli.

In trailmix_ludicrous: $Q = (Q_x, Q_y)$ is classical, so `coord_add3x(circ, x2, ox, qubits)` ($x2 \mathrel{+}= 3{\cdot}Q_x \bmod p$) costs **0 Toffoli**.

9.10. Converged-Tail Gate Elision

In the last K iterations of the 530-step GCD, the algorithm has converged and the apply-phase `cswap` has a control that is deterministically 0 on all but rare inputs. Skip the cswap for those last K iterations.

Cost: Island-exact. Requires re-hunting the tail nonce after applying.

9.11. GAP_J2 Comparator Window Narrowing (−1.33M Toffoli)

The swap decision comparator at each GCD step checks whether $|u| > |v|$. A full 256-bit comparison would be expensive, but the GCD ensures that u and v typically differ somewhere in their top bits.

The schedule `GAP_J2[i]` (258-element table) sets the comparator window width per GCD step. The comparator scans only the top `GAP_J2[i]` bits. A 22-line edit to the table to shave ≈ 1 bit per step across 258 steps:

$$\text{Toffoli saved} = 1 \text{ bit} \times 516 \text{ comparator calls} \approx 1.33\text{M Toffoli}$$

Island-exact — requires nonce hunting.

9.12. Ancilla-Free Majority

Standard carry logic uses a 3-input majority gate with 3 Toffoli and an ancilla. The identity:

$$\text{maj}(a, k, c) = c \oplus ((a \oplus c) \wedge (k \oplus c))$$

implements the same majority with **2 Toffoli and no ancilla** (XOR = CNOT, AND = CCX). Peak-neutral, value-identical, pure count reduction wherever majority gates appear.

9.13. Exact-Adder Recovery

Counter-intuitive: Sometimes a truncated adder costs **more** Toffoli than the exact full-carry adder, because the truncation requires an expensive correction to handle the rare carry-escape cases.

The rule: If the correction overhead of a truncated adder exceeds its savings from dropping bits, revert to the exact adder. Named: `DIALOG_GCD_APPLY_FINAL_TOPCLEAN=0` $\rightarrow \approx -2{,}597$ avg-Toffoli, value-exact.

9.14. Skip Redundant Final Cleanup

A folded/structured computation sometimes makes a final "top-clean" pass redundant — the structure ensures the cleanup bits are already zero by construction. Dropping the cleanup removes its Toffoli.

Highest-risk move

If the cleanup is not actually redundant, you get silent phase/classical dirt on some inputs. Always pair with strong `cls/pha/anc` validation before enabling.

9.15. Off-Peak Loosening (The Dual Move)

When a phase falls **off** the qubit peak (another phase is now the binder), you can WIDEN that phase's arithmetic to recover Toffoli for free:

```
Phase A: was binder at 1170q -> locally reduced to 1162q
Phase B: now binder at 1170q (unchanged)
Action: widen Phase A's carry width back toward the old value
Effect: Phase A's Toffoli drops; Peak stays at 1170 (Phase B is binding)
```

The rule: Shrinking an off-peak phase is a pure-Toffoli play. Widening an off-peak phase is a free Toffoli refund. Govern both on gates alone, not qubits.

9.16. Constprop: Automated Static Gate Elimination

The constraint-propagation pass (`CONSTPROP_MAX_ITERS`) analyzes the circuit statically and eliminates:

- **Provably-constant CCX gates**: control always 0 → remove; control always 1 → replace with CNOT
- **Inverse-pair cancellation**: consecutive `CCX(a,b,c)` gates that cancel algebraically
- **Affine simplification**: sequences simplifying to an affine function, implementable with fewer gates

Setting `CONSTPROP_MAX_ITERS=256` (vs default 16) runs to a deeper fixpoint, finding more opportunities: −377k Toffoli in one version.

9.17. Quadratic-Cascade → Controlled-Increment (cinc)

Replace an $O(t^2)$ gadget with an $O(t)$ one when the math allows. A recurring pattern is a "carry-into-tail" or prefix-propagation built as a **quadratic cascade of multi-controlled-X gates** (`mcx_clean_k` over a window $[nv, L)$ — every position controls on all positions below it, giving $O(t^2)$ MCX work).

When the cascade is computing a *carry propagation* (the standard "increment if all low bits are 1" pattern), it is exactly a **controlled increment**, and there is a cheaper exact primitive for that: the **Khattar–Gidney controlled-increment** `cinc` (arXiv:2407.17966), which uses only $\log^* n$ clean ancilla and $O(n)$ Toffoli instead of the quadratic cascade.

```
Old: clean-tail fold carry = mcx_clean_k prefix cascade over [nv, L)   -> O(t^2) MCX
New: clean-tail fold carry = cinc_khattar_gidney (one controlled +1)   -> O(t), log*-
    ancilla
```

Value-exact and density-neutral: an exact functional replacement (same truth table), so it does not touch the island distribution — it can be stacked freely without re-hunting a nonce. Named `TLM_FOLD_TAIL_CINC`; worth ∼−5,175 avgT alone. This is the **"safe" Toffoli lever class**: unlike dead-CCX deletion (§9.8, distribution-exact), a cinc swap is correct on all inputs — prefer this class when choosing what to stack first.

The general principle: **whenever a gadget's cost is quadratic in a window width, ask whether it is secretly a standard primitive (increment, compare, prefix-AND) with a known linear construction.**

9.18. Pseudo-Mersenne (Solinas) Modular Arithmetic

This is the workhorse that makes *every* modular operation cheap. The math background is in §4.5: because $p = 2^{256} - c$ with $c = 2^{32} + 977$ only 33 bits, $2^{256} \equiv c \pmod p$, so a reduction is a cheap **fold** of the top bits into the bottom, not a division. Here is how that identity becomes actual reversible gadgets (Schrottenloher Algs 7/10/11; trailmix `pm_prims.rs` / `arith.rs`). The constant c is called f in the code.

Modular doubling — $2x \bmod p$ (Alg 7). Shift x left by one bit (now up to 257 bits). The part that spilled past bit 256 has weight $2^{256} \equiv c$, so it folds back as $+c$:

```
2x mod p:
  1. shift-left by 1                        (a relabel -- 0 Toffoli)
  2. if the top bit overflowed: add c into the LOW limbs   (controlled +f, short)
  3. one CX to clear the overflow ancilla
```

Modular addition — $x + y \bmod p$ (Alg 10). $x + y$ overflows 2^{256} by at most one bit; on overflow, fold it as $+c$ in the low limbs, then erase the overflow flag with a compare:

```
x + y mod p:
  1. add y into x                           (a Cuccaro/Gidney adder)
```

```
2. if overflow (carry-out = 1): add c into the low limbs
3. erase the overflow ancilla via a y<x comparison
```

Alg 11 additionally handles the degenerate $x + y = p$ case (all high bits equal) — needed only in the GCD's first $\approx c\sqrt{n}$ "empty" padding iterations, so the expensive variant is routed *only* there and the cheap Alg-10 used everywhere else.

The two approximations that make it cheap (and their dials). Doing these exactly would still need full-width compares and adds. Two structural facts let you truncate — because c is small:

- $+f$ **touches only the low limbs (the `lsbs` dial).** Folding $+c$ perturbs only the bottom $\sim$33 bits plus a short carry ripple, so the $+f$ adder runs over a window of width `lsbs` (trailmix uses $\sim$54–63). Too narrow misses a long ripple; per-call failure $\approx 2^{-\text{padding}} \approx 2^{-30}$ at their settings. *Shipped as* `*_CARRY_TRUNC_W` / `ROUND84_INPLACE_SOLINAS_FOLD` / the low-54-bit $+f$ fold (§13).
- **Overflow/degeneracy decided from only the top bits (the `msbs` dial).** Whether $x + y \geq p$ (or $2x \geq p$) is dominated by the high bits, so the compare scans only the top `msbs` bits (the paper uses 40–50). *Shipped as* `DIALOG_GCD_COMPARE_BITS` / `*_APPLY_CLEAN_COMPARE_BITS`.

Both are **graded / island-exact** levers (§6, §11): tightening a notch is value-exact only on a searched nonce-island, because it makes a 2^{-k}-rare fraction of inputs reduce wrong — exactly the binary-vs-graded line of §6, tuned against the 9024-shot 0/0/0 target.

Borrowed-dirty $+f$ window (`gidney_cadd_f_window`). The $+f$ add needs carry scratch; instead of fresh ancilla it **borrows the register's own idle high bits** ($\sim$3 clean ancilla instead of $\sim$10), restoring them after — gate-hosting (§7.16) applied to the fold. The reduction adds $\sim$0 clean qubits at the peak, value/phase-identical to the vented path.

Why it matters. A generic-prime reduction is a full multiprecision divide (Barrett/Montgomery, $\sim$n-bit work) on *every* add and double, and the GCD/Bézout reconstruction does thousands of them. Pseudo-Mersenne turns each into a short $+f$ fold + a top-bits compare — the entire secp256k1 advantage (constant-adder 15% vs 34%, modular-double 8% vs 24%, §4.5). **No Montgomery form is used** — both the paper and trailmix keep plain integer representation and fold directly. NAF recoding (§9.7) is the next layer: it makes the fold constant c itself cheaper (4 signed terms instead of 6).

9.19. Merging Adjacent Controlled-Swaps

The GCD loop swaps its two operand registers conditionally at almost every step (the "is $|u| > |v|$?" decision swaps u and v, and likewise their Bézout cofactors). A controlled swap of two n-bit registers costs n Toffoli (one `cswap` per bit). Across $\sim$530 steps $\times$ two register pairs, these swaps are a large slice of the Toffoli budget.

The identity that merges them. Two controlled swaps of the *same pair of registers*, controlled by bits p and q, compose into a *single* controlled swap controlled by $p \oplus q$:

$$\texttt{cswap}(p, A, B) \cdot \texttt{cswap}(q, A, B) \;=\; \texttt{cswap}(p \oplus q, A, B).$$

(Swapping under p, then again under q, swaps exactly when an odd number of the two controls fired — i.e. when $p \oplus q = 1$.) So wherever two swap decisions on the same registers sit next to each other — e.g. the cofactor swap at the *end* of one GCD step and the operand swap at the *start* of the next — you replace **two** n-Toffoli swaps with **one**, after spending a single cheap CNOT to compute $p \oplus q$ into a "frame-parity" qubit that carries the accumulated swap state across the boundary.

This is **algebraic fusion (§9.4) specialized to swaps**, and it is value-exact (a pure identity, all inputs). It was the single biggest Toffoli win of the early era — the Kaliski `step9∘step3` boundary merge, −274,000 **Toffoli, peak-neutral**. The general lesson: *controlled swaps compose through XOR of their controls* — look for adjacent swaps on the same lanes and collapse them.

10. The Qubit ↔ Toffoli Exchange Rate and the Product Race

10.1. The fundamental arithmetic

At the current SOTA (1152q × 1,364,230 avgT = 1,571,592,960 score):

$$\text{break-even rate} = \frac{\text{avgT}}{\text{peak\_q}} = \frac{1{,}364{,}230}{1152} \approx 1{,}184 \text{ Toffoli per qubit}$$

A lever that saves 1 qubit at the cost of fewer than 1,184 Toffoli **improves the score**. A lever that costs more than 1,184 Toffoli per qubit saved **worsens the score** — even though it reduced qubits.

This rate **changes with every optimization**:

- After a large Toffoli reduction: the rate drops (qubits become more valuable). Previously-marginal qubit cuts are now worth trying.
- After a large qubit reduction: the rate rises. Previously-marginal Toffoli cuts might now dominate.

10.2. The product-race caveat

A qubit drop that clears break-even can STILL lose the product race if the Toffoli-grinding track moves faster in parallel.

Concrete example (June 2026): A 1157q clamp cost ≈ 1,127 Toffoli/qubit — below break-even at landing. But the 1159q Toffoli-grind reached 1,378,242 avgT. Score comparison:

$$1159 \times 1{,}378{,}242 = 1{,}597{,}506{,}378 < 1157 \times 1{,}380{,}890 = 1{,}598{,}290{,}330$$

The 2-qubit savings was overtaken by the parallel Toffoli-cutting track.

Lesson: Always run both tracks and compare products. Never minimize qubit count in isolation.

10.3. The shelved-lever rule

A qubit lever that **failed break-even on one Toffoli base** can become favorable on a cheaper Toffoli base.

Example: The dynamic headroom clamp (`TLM_TARGET_Q`) cost ≈ 1,514 Toffoli/qubit on the schoolbook-square base. After Karatsuba reduced the square cost, the same clamp cost only ≈ 20 Toffoli/qubit on the new base — well below break-even. It won.

Rule: After any major arithmetic win (Karatsuba, algebraic fusion, structural rewrite), re-test **all previously-shelved qubit levers** at the new exchange rate.

10.4. Different peak layers have different exchange rates

At any peak, there are typically two layers:

1. **Persistent floor**: long-lived state held across the whole peak (transcript, passenger values, GCD state). Often **cheap** to reduce — a denser codec or transcript streaming costs few gates to save many qubits.
2. **Transient working registers**: the current step's carry bits, arithmetic scratch. Often **expensive** to reduce — freeing them requires recompute/dirty-borrow, adding many Toffoli.

Counter-intuitively: **compress the transcript before freeing the scratch**. The transcript looks "irreducible" (structured data) but is often the cheap lever. The scratch looks "wasteful" but is often the expensive lever.

11. Density-Neutral vs Island-Exact — Correctness Regimes

11.1. The fundamental distinction

- **Density-neutral (value-exact)**: The circuit produces the same output on ALL inputs. The optimization is grounded in mathematical identity or circuit structure.
- **Island-exact (distribution-specific)**: The circuit is correct only on the specific 9024 test inputs. On other inputs, it may give wrong answers.

When you apply an island-exact optimization, you need to find a new **nonce** — a 48-byte seed for the test input generator — such that all 9024 generated inputs happen to be in the safe zone. Stacking multiple island-exact optimizations makes the safe zone smaller.

11.2. A common island-exact lever: active-iteration trimming

The GCD inversion is run for a fixed number of steps chosen to cover the *worst-case* input. But the worst case is rare, so you can run **fewer** iterations (`DIALOG_GCD_ACTIVE_ITERATIONS`, e.g. 399 → 393) and accept that a $\sim 2^{-k}$-rare fraction of inputs won't have fully converged. This is a pure island-exact lever — it drops whole divsteps (qubits *and* Toffoli) but makes the circuit wrong on those rare inputs, so it must be paired with a nonce hunt. It sits alongside carry-truncation (§7.4) and comparator-narrowing (§9.11/§9.18) as a "graded" knob (§6).

The line between island-exact and cheating

Island-exact levers are legitimate *because the 9024 validated inputs are the actual scored distribution* and the failures they introduce are genuinely rare and structurally understood. There is a real line, though: deliberately truncating the circuit so it is wrong on a *large* fraction of inputs and then re-rolling the nonce purely to *hide* those failures is verifier-gaming, not optimization — the early history contains at least one flagged "red-team" example (a 396-iteration build wrong on $\sim 3 \times 10^{-4}$ of inputs, shipped with a comment admitting it). The community norm: an island-exact cut should be a structurally-sound approximation whose error you can *bound and explain*, not a way to launder a broken circuit past the gate.

11.3. The 6dafa07 pivot: empirical → structural dead-CCX

Before the current SOTA, the circuit used **empirical dead-CCX elimination**: sample 9M inputs, find CCX gates that never fire, skip them. Island-exact.

The `6dafa07` commit replaced this with **structural dead-CCX skipping**: prove from circuit structure which CCX gates can never fire on *any* input. These predicates are keyed

by call-site and bit-index, not sampled data. Density-neutral.

11.4. The correctness check triple: `cls/pha/anc`

Every validation run checks three error types:

- `cls`: classical mismatch (the output value is wrong on some input)
- `pha`: phase error (the output is correct but entangled with extra phase — kills interference)
- `anc`: ancilla mismatch (a qubit failed to uncompute back to $|0\rangle$)

A correct circuit shows `0/0/0`. Measurement-based uncompute fails in `pha` (which value checks miss). Truncation failures typically show in `cls`. Missed uncompute shows in `anc`.

12. Pebbling Theory — The Formal Foundation

12.1. The reversible pebble game

The **pebble game** models space-time tradeoffs in reversible computation. The computation is a directed graph (nodes = values, edges = dependencies). A "pebble" = a qubit holding that value. Peak qubits = maximum pebbles simultaneously.

12.2. Bennett pebbling: classical reversibility is expensive

Bennett (1989) showed that reversibly simulating a T-step computation using S extra pebbles costs:

$$\text{Time} = \varepsilon \cdot 2^{1/\varepsilon} \cdot T^{1+\varepsilon}/S^{\varepsilon} \quad (\text{for any } \varepsilon > 0)$$

The hidden constant $c(\varepsilon) = \varepsilon \cdot 2^{1/\varepsilon}$ grows **exponentially** as $\varepsilon \to 0$.

Concrete example: Halving the GCD transcript (S halved, $\varepsilon = 1$):

$$\text{Time blowup} \approx 1 \times 2 \times 530^2/370 \approx 1{,}518 \text{ extra GCD passes}$$

At $\approx$ 2,500 Toffoli per pass: 3.8M extra Toffoli for $\approx$ 370q savings. Break-even needs: 3.8M/1,184 $\approx$ 3,200 qubits. Actual saving: 370. Not worth it.

12.3. Quantum (spooky) pebbling: exponentially better

Gidney (2019) introduced **spooky pebbling**: also allow removing a pebble by measuring it in the X-basis, recording the measurement outcome as a classical bit. When the value is needed again, recompute it and apply a classically-conditioned phase correction.

Kornerup, Sadun, Soloveichik (2021) proved tight bounds:

$$\text{Quantum:} \quad T_{\text{quantum}} = O(T/\varepsilon) \quad [\text{polynomial constant}]$$
$$\text{Classical:} \quad T_{\text{classical}} = O(2^{1/\varepsilon} \cdot T) \quad [\text{exponential constant}]$$

The quantum advantage is exponential. This is the theoretical backbone of why MBU and measurement-based techniques dominate.

12.4. Parallel spooky pebbling (Kahanamoku-Meyer et al. 2025)

For a length-ℓ sequential computation chain:

$$\text{qubits needed} = 2.47 \times \log(\ell) \quad \text{at depth } 2\ell$$

For our 530-step GCD: $2.47 \times \log(530) \approx 23$ qubits. Our current GCD state machine uses $\approx$ 22 qubits — already near the theoretical minimum.

12.5. How the optimization techniques *are* pebble-game moves

This theory is not background decoration — **every qubit-freeing technique in §7–§9 is literally a move in one of these two pebble games.** Naming the move tells you its exact cost and risk:

Technique	Pebble-game move	Which game	Cost to un-pebble
Live-range hole (§7.1)	remove a pebble by **uncompute**, re-place later by **recompute**	Bennett	full Toffoli, *twice*
Free-and-recompute cy0 (§7.5)	same, on one cheap pebble	Bennett	≈0 (a few CNOTs)
MBU / carry vent (§9.1–B)	remove a carry pebble by **X-measurement + CZ**	spooky	**0 Toffoli**
Conditional replay (§9.3)	measure the pebble; recompute only on the firing branch	spooky	Pr[fire]× cost
HMR ghosting (§7.14)	measure, leave a **ghost**, recompute + phase-fix	spooky	one recompute
Transcript compression (§7.8)	**don't pebble it** — shrink the data so fewer pebbles persist	(out of the game)	codec only
Parallel GCD schedule (§13)	optimal pebble *placement* over the 530-step chain	parallel spooky	— (already optimal)

The Bennett→spooky upgrade is the single biggest constant-factor lever. A live-range hole (§7.1) is a *Bennett* move: to free a pebble you uncompute it (paying its Toffoli) and later recompute it (paying again) — the classical-reversible game, where un-pebbling is expensive. MBU (§9.1) is the *spooky* move on the same kind of pebble: you remove it by **measuring** it, and a measurement costs no Toffoli. That is exactly why an n-bit adder drops from ∼2n Toffoli (compute + coherent uncompute) to ∼n (compute + measured uncompute) — **MBU is the pebble game's quantum advantage cashed out on every carry bit.**

Pebbling theory also tells you *which* qubits to attack. The recompute-to-free move only pays when `recompute_Toffoli` < `qubits_freed` × `break_even` (§10). The two extremes:

- **The transcript is firmly in the "never recompute" regime.** Bennett's exponentially-growing constant is why: re-deriving it costs ∼3.8M Toffoli to free ∼370 qubits (∼8× over break-even). So take it *out* of the game by **compressing the data** (§7.8), not by recomputing it.
- **A scratch pebble like `cy0` is deep in the "always recompute" regime.** It is a cheap function of live qubits, so recompute costs ∼0 Toffoli — free it without hesitation (§7.5).

And the parallel-spooky bound says the **GCD control chain is already done** (≈22q vs the $2.47 \cdot \log(530) \approx 23$q floor). So the theory aims every remaining qubit lever at the **data pebbles** — transcript, passengers, coordinate registers — exactly where §7 and §13 spend their effort. In short: *recompute the cheap pebbles (Bennett), measure the carry pebbles (spooky), compress the expensive data instead of pebbling it, and stop optimizing the control chain.*

13. The SOTA Circuit: How trailmix_ludicrous Works

13.1. The decisive architectural choice: Q is classical

The second point $Q = (Q_x, Q_y)$ is kept as classical `BitId` values, not quantum registers. Materialized into a transient quantum temp *only* at off-peak coordinate steps, then freed. This eliminates 512 qubits from the peak.

13.2. Two GCD passes sharing one tape

The affine point addition needs:

1. $\lambda = dy \cdot dx^{-1} \bmod p$ — inverse GCD (drive $(p, dx) \to (1, 0)$)
2. new_y_component $= \lambda \cdot (P_x - P'_x) \bmod p$ — forward GCD

Both share **one** modular-inverse engine and **one** dialog tape (transcript). This avoids having two separate inversion circuits, saving both qubits (one shared tape) and Toffoli (shared codec overhead).

13.3. The base-5 codec: 772 → 603 qubits live

At each of 257 divstep steps, the algorithm records (`subtracted`, `swap`, s_2) — 3 bits raw. But only 5 of 8 possible 3-bit patterns are reachable. Triple windows have $5^3 = 125$ reachable states, encoded in 7 bits instead of 9:

```
Layout: 1 Step0 (2b) + 85 Triples (7b each) + 1 Pair tail (5b) + 1 t1 flag (1b) = 603
    qubits
```

13.4. The vented-adder zoo

Every adder in `trailmix_ludicrous` vents its carry-uncompute via MBU:

```
hmr(carry, bit);       // X-basis measurement, 0 Toffoli
cz_if_bit(a, b, bit);  // phase correction conditioned on measurement
```

The adder family dispatches per-call by baked schedule tables. Each call's vent budget: exactly `TLM_TARGET_Q` − `active_qubits`, spending every spare qubit below the ceiling on Toffoli savings.

13.5. The deliberate island-exact components

trailmix_ludicrous includes calibrated island-exact approximations:

- **Low-54-bit +f fold** (`LSBS = 54`): reduction folds touch only low 54 bits; carry beyond bit 53 is dropped with probability $\approx 2^{-21}$ per fire.
- **Narrow-top-window comparators** (`MSBS = PAD = 21`): swap predicates scan only the top 21 bits. Mis-decision probability $\approx 2^{-21}$ per call.

13.6. Where 1152q comes from

At the SOTA (`d44cad3`):

- **603q**: compressed base-5 GCD transcript (live during both GCD passes)
- **512q**: two 256-bit coordinate registers (quantum P_x, P_y)
- $\approx$ **37q**: transient carry/vent ancilla at the peak-binding fold operations

14. The Three Tracks: Score, Low-Qubit, and Pareto Frontier

The challenge has three distinct objectives (§2.4), and each has its own history and its own winning constructions. This section walks all three: the **score track** (minimize Q×T), the **low-qubit track** (minimize Q alone), and the **Pareto-frontier track** (map the clean (Q, T) curve).

14.1. The score track (Q×T): the 2715q→1152q journey

Pre-history: 2715q → 1211q (where the circuit came from)

The challenge *started* at a baseline of **2,715 qubits** × **∼3.96M Toffoli**. The journey down to the 1211q dialog-GCD circuit (where the table below begins) is its own story, and it shows the same methods appearing in cruder form:

- **The baseline** was a **Roetteler-style two-Kaliski affine** point addition: affine coordinates (not projective), the modular inverse done by *two* runs of the Kaliski binary-GCD algorithm, with Q and the prime kept **classical from the start** and direct/Solinas reduction (no Montgomery). The inverse was ∼80–90% of the Toffoli — the bottleneck §4 names.
- **2708 → ∼2002 (scratch-packing on a fixed inverse).** With the Kaliski engine fixed, the early drops were *all* live-range and hosting tricks: the `cswap`$(p \oplus q)$ swap-merge (§9.19, −274k Toffoli), borrowing **provably-$|0\rangle$ high bits** of the shrinking GCD register as carry scratch (zero as a classical function of the iteration index — gate-hosting §7.16 in its first form, reaching the published "$9n$" peak floor), square-recompute eviction (§7.1), and **joint-pin plateau breaking** (§7.9) — the realization, present from the very start, that a peak co-owned by several scratch clusters drops only when you reduce them all *together*.
- **2002 → 1698 (the engine swap).** The first true *architectural* jump replaced the entire Kaliski stack with a **dialog-GCD inverter that streams a compressed transcript** instead of holding wide history registers resident (§4.3) — one change worth −304 qubits, and the lineage the SOTA still descends from.
- **1698 → 1211 (windowing + recompute-to-free).** On the dialog-GCD engine the peak fell via the **windowed/chunked apply** dial (§7.18, the era's biggest peak lever), carry self-hosting on the operand lanes (§7.13/§7.16), shift-free modular doublings (§7.19), keep-live-vs-recompute in the Solinas fold (the keep-alive dual, §7.1), and the first **recompute-to-free fold carries** — the same family as the cy0 trick (§7.5) that later broke 1153→1152.

So nearly every technique in this primer was *already in play* before 1211q, just on a coarser circuit. The table below picks up at 1211q, where the modern dialog-GCD/trailmix line begins.

The 1211 → 1152 ladder

Each qubit drop used the technique that fit its specific bottleneck:

Transition	**Technique**	**What was released**
1211→1193 (−18q)	Live-range hole (§7.1)	GCD-apply scratch block freed during shift sub-phase; reacquired after
1193→1192 (−1q)	Truncation (§7.4)	Square segment one notch tighter
1192→1185 (−7q)	Transcript codec (§7.8)	Per-step fold carry scheduling + exact 11-bit head codec
1185→1170 (−15q)	Plateau decomp (§7.9) + 5 techniques	Tail codec, streaming apply, square rebalance, 5 co-binders
1170→1169	MBU vent on fold carry (§9.1/8.B)	`hmr+cz_if` on the $-f$ fold peak owner: 0 Toffoli to vent it
1167→1166	Step-0 swap_flag elimination	`swap_flag` becomes `Option<QubitId>`, lazy-alloc; freed one tape lane
1166→1165	Odd-u0 parking (§7.1)	`u[0]` is provably 1; park+loan the lane, restore in 2 gates
1164→1163	Source-as-carry suffix	Paid drop: surrendered apply-phase vents to free one carry lane
Toffoli: −22.4M	Karatsuba square (§9.6)	3 sub-squares instead of 4
Toffoli: −1.33M	GAP_J2 narrowing (§9.11)	22-line per-step comparator table edit
Toffoli: −377k	Constprop deeper (§9.16)	`CONSTPROP_MAX_ITERS` 16 → 256
1163→1159→1156	Headroom clamp returns (§7.10 + shelved-lever §10)	Same clamp that lost early now wins on the cheap Karatsuba+dead-CCX base (∼527 T/qubit)
1156→1153	Clamp + zero-cin fold + FFG cap (§7.10), paid by cinc (§9.17) + iterated dead-CCX (§9.8)	Qubit drop alone cost +11,369 avgT (rejected); Toffoli levers stacked back to win
1153→1152	Free-and-recompute cy0 (§7.5)	`cy0 = ctrl & !a0`; loan lane for 40 binding folds
Structural	6dafa07 pivot (§9.8/§11)	Replaced empirical dead-CCX with ≈ 15 structural predicates

Key meta-lessons:

1. **Every "floor" fell to a better encoding or a better question.** Not always a new algorithm — often a denser codec or the question "whose value is a cheap function of live qubits?"
2. **The shelved-lever rule is real.** The 1157q clamp tried early and lost; won after Karatsuba.
3. **Plateau mode is the endgame.** The final qubit drops each required coordinated packages of 4–9 independent local reductions applied simultaneously.

4. **Structural correctness wins over island-exact tricks.** The 6dafa07 pivot to structural dead-CCX eliminated the empirical approach entirely — correct for all inputs, not just the 9024.

14.2. The low-qubit track: the Shrunken-PZ inversion (§2.4, track 2)

What the Shrunken-PZ approach is (the backbone of this track)

Almost every record on the low-qubit track is built on one architecture, **Shrunken-PZ** (Proos–Zalka). It is worth understanding as a whole, because it is a *fundamentally different inversion engine* from the ludicrous/dialog-GCD circuit that holds the product SOTA — different module, different objective, sharing only the problem statement ($P \mathrel{+}= Q$).

The core idea. Where the ludicrous track records a small **decision tape** and replays it (§4.3 dialog transcript), Shrunken-PZ runs the **Proos–Zalka binary-GCD / Kaliski divstep inversion directly as a bit-by-bit reversible state machine** — no big transcript. It carries the actual Euclidean state in **five working registers** and *resizes them every step*:

register	meaning	width over the 530 steps
A, B	the two running GCD magnitudes	start 256, **shrink toward 1** as the GCD converges
CA, CB	the two Bézout cofactors (inverse accumulators)	start 1, **grow toward 256**
Q	the per-step quotient	small, ∼21–25 bits

Why the peak is in the middle, and why register sharing fits. A, B shrink while CA, CB grow, so the qubit peak is not at either end — it is at the **GCD crossover** (∼step 363), where both pairs are mid-sized:

$$\text{peak} \approx 2\max(|A|,|B|) + 2\max(|CA|,|CB|) + |Q| + \text{FIXED}(\sim 267).$$

At the crossover this is $[81, 81, 248, 248, 23] = 681$ working bits + ∼267 fixed (the 256-bit coordinate passengers, modulus, sign/parity, counter, comparator scratch) = **948 qubits**. Crucially A and its cofactor CA are **complementary in size** (one shrinks exactly as the other grows) — which is what makes EEA register sharing (§7.15) able to pack $A \| CA$ into one word and push toward 851/829.

The levers that define the track (each is a §7 technique, applied to the divstep machine):

- **Dynamic-width registers (§7.6)** — resize A, B, CA, CB per step; free known-zero high bits.
- **CLZ-context window narrowing** — the divstep arithmetic only touches bits above a provably-zero low region, so each op runs over a narrowed window $[lo..len)$. The "robust width envelope" projects the tightest per-step window that still covers all sampled inputs — the qubit-side twin of empirical dead-CCX (§9.8), and equally **island-exact**.
- **Known-constant teardown (§7.7)** — at convergence $A{=}0, B{=}1, CA{=}p, Q{=}0$ are constants; XOR them out (0 Toffoli) before the λ multiply so they don't co-reside with it.
- **Passenger ghosting (§7.14)** — HMR-ghost dy/λ so only one 256-bit coordinate co-resides with the EEA peak (+256, not +512).
- **Pipelined two-quotient divstep** — build the *next* quotient while draining the *previous*, so the quotient record stays ∼26 bits instead of a full 256-bit tape.
- **EEA register sharing (§7.15)** — the deepest lever; packs operand and cofactor, reaching 851/829q.

Why it is the qubit backbone but not score-competitive. Running full per-step modular arithmetic *without* the transcript-replay and Karatsuba machinery makes the divstep inversion inherently gate-heavy: ∼33–55M Toffoli at 948–980q (∼40× the ludicrous SOTA's 1.36M), and ∼460M at 851q. So Shrunken-PZ maps the **qubit floor** (objective 2) but loses the **product** by a wide margin — a qubit-lower-bound witness, not a contender on score.

The record ladder

Running *alongside* the Q×T product-SOTA journey above is this separate **low-qubit competition** line — the Shrunken-PZ family — whose objective is to minimize peak qubits with Toffoli unconstrained:

Qubit record	Technique	Notes
1050q	trailmix shrunken-PZ embedded op stream	early low-qubit route
1019q	source-level dynamic-width PZ port	
988→956q	dirty-borrow / gate-hosting lever stack	
952→948q	further lane borrowing + thin schedule	lowest *fully-validated* witness
851q (`e7dd3de`, 6/23)	**EEA register sharing (§7.15), Luo 2026**	**−97q; the breakthrough**
829q (`1dd61ca`, 6/24)	more borrowing/relocation on a frozen envelope	current low-qubit frontier

These are records on the **pure-qubit objective**, where the ∼400–500M Toffoli they spend simply does not enter the scoring. The 851q and 829q figures come from an analysis-oracle accounting (§7.15) — read them as the leading **lower-bound witnesses** for the qubit floor; the 948q route is the lowest fully-validated circuit on the track. This is a *different game* from the Q×T product SOTA (1152q × 1.36M, value-exact): more qubits but ∼340× fewer Toffoli. The two minima are reached by different constructions, and which one is "best" depends entirely on which competition you are entering — see §7.15 and §10.

14.3. The Pareto-frontier track: clean (Q, T) basis circuits 1153q → 1133q (§2.4, track 3)

The third track (§2.4, objective 3) maps the **(Q, T) Pareto frontier** below the 1153q SOTA as a sequence of **clean, dead-CCX-free basis circuits** others can fork. "Dead-CCX-free" is the precise claim: every rung hard-sets `DROP_DEAD_ROBUST_DISABLE=1`, omitting the most overfit lever (whose hard-coded gate-index `.idx` lists are tied to one exact op stream, §11) — that omission is what keeps the basis *reusable*. They are **not fully value-exact**, however: each still carries the route's standard calibrated **approximations** (comparator-width narrowing §9.18, carry truncation §7.4, and at the lowest rungs **active-width trimming** §11) that are island-exact and require a per-rung nonce hunt. So treat the curve as a *cleaner, more forkable* reference than a dead-CCX-stacked SOTA, not as an all-inputs-provable bound.

Unlike the low-qubit witness track above, these stay in the **useful corner** — every point beats the published *single point-addition* estimate on *both* axes: $q < 1175$ and $T \approx$ 1.37–1.46M, comfortably under the ∼2.69M ($2^{21.36}$) Toffoli of the Babbush space-optimized secp256k1 point-add (Schrottenloher 2026, Table 1). So the whole curve clears the bar with ∼20–40 qubits and roughly 2× Toffoli to spare. They are "rejected" on the Q×T product *by*

construction (they trade qubits down for Toffoli up); the deliverable is the exchange *curve*, not a score.

q	T_{clean}	**exch. vs prev**	**new lever introduced**
1153	1,392,603	— (anchor)	dead-CCX off baseline
1147	1,396,769	−694 T/q	fold-vent clamp (`TLM_FOLD_CALL_CODE_OVERRIDES`)
1146	1,408,582	−11,813 T/q	graduated→dirty carry-in (`TLM_GRAD_DISABLE`)
1143	1,411,643	−1,020 T/q	codec scratch reuse + direct-dirty fold ctl
1142	1,415,790	−4,147 T/q	pair-raw codec last-k + tighter layout search
1141	1,423,723	−7,933 T/q	no-ancilla/dirty body + constant-lane loan family
1133	**1,460,511**	−4,599 T/q	**GCD active-width trimming** + fold-cout drop

The qubits sort by exchange rate into four lever classes: **fold-vent clamping** (the cheap qubits — clamp a peak fold's measured-vent windows, §9.2), **no-ancilla / dirty-borrow substitution** (convert a carry ancilla into Toffoli, cf. §7.13), **constant-lane loan + recompute** (the Bennett pebble move on provably-constant GCD low bits, §7.1/§12), and — new at 1133 — **GCD active-width trimming**: per-step shrinking of the live GCD operand width in the convergence tail (`TLM_GCD_ACTIVE_WIDTH_TRIM=3` after step 205), the dynamic-width-register idea (§7.6) applied to the ludicrous GCD. The 1133 rung is a *value-exact repair* of an over-aggressive 1131 attempt: trimming the active width is a **graded** correctness lever (§6) — push until the residual breaks, then give back one notch (1131→1133) so a clean nonce is reachable. The push also introduced a reusable general pass — **reset-bounded qubit-id compaction** (§7.17). Full analysis: `pareto-frontier-push-1153-to-1133.md`.

15. Open Frontiers

15.1. Density-neutral Toffoli cuts not yet applied

1. **Gidney 2025 streaming vented adder** (arXiv:2507.23079): Uses 2 clean + $(n-2)$ dirty ancilla for $\approx 3n$ Toffoli. Partially implemented in `venting.rs` but **leaks phase** — dirty ancilla phase corrections are incomplete. Expected $\approx n/4$ Toffoli savings per adder once fixed.
2. **Apply-swap structural dead-CCX**: The GCD apply-step `cswap` block (≈ 256 cswap per iter $\times$ 258 iters $\times$ 2 passes) has NO per-bit structural-dead skip. Porting the structural-dead analysis to these operations is a high-priority open lever.
3. **Extended Cuccaro structural dead-carry**: Currently checked for call indices 13–37 only. Extending to ALL Cuccaro adder call sites would save proportionally more.
4. **Apply-swap involutory pair cancellation**: The forward GCD-swap and the immediately following GCD-cswap on the same (u/v) lanes may cancel algebraically (swap twice = identity). Not yet checked structurally.
5. **Windowed arithmetic / QROM lookup** (Gidney 2019, arXiv:1905.07682) — *the biggest untried structural lever.* A multiply-by-a-quantum-value normally does 2^w controlled add-shifts. The windowed trick reads a w**-bit window** of the multiplier and uses

it to **address a QROM table lookup** that adds the precomputed partial product in one shot — turning 2^w controlled adds into one table read. The Bézout-reconstruction multiplies (§4.3) are ∼90% of the Toffoli, so a windowed/QROM form there is the largest single cut on the board. Open question: whether the value-dependent reconstruction multiply can be windowed the way the fixed-base $k \cdot G$ scalar mult already is.

15.2. Density-neutral qubit cuts not yet applied

1. **The co-bind plateau at 1152q**: Multiple independent phases all tie at 1152q. Each needs a local reduction before the global peak drops.
2. **FFG cy0-style recompute on other binders**: The cy0 trick (§7.5) worked because `cy0` was a cheap function of live qubits. Other binder qubits may have similar cheap-function properties not yet identified.
3. **SumHiLo/shifted vents** (`TLM_SQUARE_SUMHILO_VENT`, `TLM_SQUARE_VENT_SHIFTED`): Exist in the codebase as default-OFF options. Off-peak phases could enable these for free Toffoli savings.

16. Quick-Reference Summary Tables

16.1. Qubit reduction techniques

Technique	**Qubits saved**	**Toffoli cost**	**Density-neutral**
Live-range hole (§7.1)	k qubits	$+2 \times C_V$	Yes
Passenger relocation (§7.2)	k qubits	0	Yes
Range-bound guard removal (§7.3)	1 per register	0	Yes
Truncation (§7.4)	varies	0	Partial
Free-and-recompute cheap value (§7.5)	1	≈ 0 (CNOTs)	Yes
Dynamic-width registers (§7.6)	varies	small	Partial
Known-constant teardown (§7.7)	k	0 (CNOT = Clifford)	Yes
Transcript compression (§7.8)	varies	decode overhead	Yes
Plateau decomposition (§7.9)	1 (global)	combined cost	Yes
Dynamic headroom clamp (§7.10)	1 per ceiling drop	small	Yes
Lazy carry liveness (§7.11)	1/chunk	0	Yes
In-place decode (§7.12)	varies	codec overhead	Yes
Borrowed-carry fusion (§7.13)	k	0	Yes
HMR ghosting (§7.14)	256	+1 modular multiply	Yes
EEA register sharing (§7.15)	∼97–119 (to 851→829q)	high (irrelevant to low-q objective)	Yes — low-qubit competition
Gate-hosting (§7.16)	1 per hosted gate	0 (clean) / +1 Tof (dirty)	Yes if host-zero *proven*; island-exact if *sampled*
Reset-bounded id compaction (§7.17)	gap (max-id − true peak)	0	Yes (relabels wires only)
Windowed/chunked materialization (§7.18)	many (the `F_CUT` dial)	+few (boundary cmps)	Yes
Shift-free modular doublings (§7.19)	∼24 (drops spill+flags)	+few	Yes
GCD active-width trim (§7.6 applied)	varies (deep tail)	small	Partial (graded; nonce-hunted)

16.2. Toffoli reduction techniques

Technique	**Toffoli saved**	**Qubit cost**	**Density-neutral**
MBU / carry vent (§9.1/2)	1 per vent	+1 temporary	Yes
Conditional replay (§9.3)	$(1-f)\times$ block	0	Yes
Algebraic fusion (§9.4)	varies	0	Yes
Witness-first dataflow (§9.5)	$\approx$ 13M / instance	0	Yes
Karatsuba square (§9.6)	$\approx$ 22M (one change)	0	Yes
NAF recoding (§9.7)	$\approx$ 5–10% of fold cost	0	Yes
Pseudo-Mersenne reduction (§9.18)	huge (15% vs 34% of CCX)	0	Yes (fold); Partial (lsbs/msbs trunc)
Merge adjacent controlled-swaps (§9.19)	n per merged pair ($-$274k once)	0	Yes
Structural dead-gate skip (§9.8)	varies	0	Yes
Classical constant ops (§9.9)	significant	0	Yes
Converged-tail elision (§9.10)	hundreds/iter $\times K$	0	No
GAP_J2 narrowing (§9.11)	$\approx$ 1.33M / 22-line edit	0	No
Ancilla-free majority (§9.12)	1 per majority	0	Yes
Exact-adder recovery (§9.13)	$\approx$ 2,600/call	0	Yes
Redundant cleanup skip (§9.14)	varies	0	Risky
Off-peak loosening (§9.15)	free refund	0	Yes
Constprop deeper (§9.16)	$\approx$ 377k	0	Yes
Cinc cascade replacement (§9.17)	$\approx$ 5,175 ($O(t^2)\rightarrow O(t)$)	0	Yes
Empirical dead-CCX drop (§9.8)	$\approx$ 13,000+ (1st pass)	0	No (island-overfit)
Iterated 2-pass dead-CCX (§9.8)	$\approx$ 2,400/extra pass	0	No (island-overfit)

16.3. Key theoretical results

Result	Source	Insight
MBU: uncompute AND for 0 T-gates	Jones 2013 (Phys Rev A); Gidney 2018 (arXiv:1709.06648)	X-measurement + CZ = free uncompute
1 Toffoli = exactly 7 T-gates	Gosset et al. 2013 (arXiv:1308.4134)	Tight lower bound
Bennett classical pebbling	Bennett 1989 (SIAM J. Comput. 18:4)	Classical: exponential constant in space savings
Quantum spooky pebbling	Kornerup, Sadun, Soloveichik 2021 (arXiv:2110.08973)	Quantum: polynomial constant (exponential advantage)
Parallel spooky pebbling	Kahanamoku-Meyer et al. 2025 (arXiv:2510.08432)	$2.47 \times \log(\ell)$ qubits for ℓ-step chain
Conditionally-clean ancilla	Khattar & Gidney 2024 (arXiv:2407.17966)	$3n$ Toffoli MCX with $\log^* n$ clean ancilla
secp256k1 *full ECDLP* frontier	Babbush, Gidney et al. 2026 (arXiv:2603.28846)	$\leq$ 1200q / $\leq$ 90M Toffoli for the whole Shor run ($\sim$28 windowed PAs)
secp256k1 *single point-add* (the challenge unit)	Babbush space-opt, via Schrottenloher 2026 Table 1 (arXiv:2606.02235)	**1175q / $2^{21.36} \approx$ 2.69M Toffoli** (the "useful corner"); Schrottenloher space-opt = 1192q / $2^{21.19} \approx$ 2.39M
Karatsuba algorithm	Karatsuba & Ofman 1962	$O(n^{1.585})$ vs $O(n^2)$ for multiplication
EEA register sharing (origin)	Proos & Zalka 2003 (arXiv:quant-ph/0301141)	Inversion dominates point-add space; operand+cofactor packing
Compact exact reversible inversion	Luo et al. 2026 (arXiv:2604.02311)	$3n + 4\lfloor \log_2 n \rfloor$ qubits (1333q, n=256) via bit-length-tracked sharing
DQI "Dialog" in-place EEA multiply (§4.3)	Khattar, Shutty & Gidney 2025 (arXiv:2510.10967)	construct/reconstruct split; $(y, 0)$ seeding fuses inverse+multiply, no explicit inverse
Windowed arithmetic / QROM (§15)	Gidney 2019 (arXiv:1905.07682)	2^w controlled adds $\rightarrow$ one w-bit-window table lookup

See also — these detailed reference docs live in the GitHub repo `github.com/jieyilong/ecdsafail_skills`, under `ecdsafail-circuit-optimization/references/`:

- `density_neutral_tradeoffs.md` — detailed technique catalog with exchange rate math
- `gidney-techniques.md` — Gidney's full lever catalog with blog/paper pointers
- `external-literature-2000-2026.md` — broader academic literature map
- `REPORT_1168_wall_revamp.md` — the trailmix_ludicrous introduction
- `frontier-1211-to-1170.md` — detailed 1211→1170 step-by-step record
- `pareto-frontier-push-1153-to-1133.md` — the (Q,T) Pareto-frontier push (§14.3)
- `SHRUNKEN_PZ_q948_track.md` — the low-qubit Shrunken-PZ track (§2.4)

Note on code paths. Source files named in this primer (e.g. `arith.rs`, `fused.rs`, `register_shared_eea.rs`) refer to the **ecdsa.fail challenge circuit** — the `trailmix_ludicrous` / `trailmix_port` modules — pulled per-submission via the `ecdsafail` CLI (`ecdsafail reset <submission>`); they are *not* part of this docs repo.